음성 기반의 사용자 맞춤형 견과류 배식 협동로봇 시스템

(협동 -2) AI(Computer Vision) 기반 협동 로봇 작업 어시스턴트 구현 프로젝트

A 그룹 2 조

[조원] 허찬영 , 김기현 , 이승준 , 최성아 , 최진우
[튜터] 이충현 , 이정현

목 차

01 **프로젝트 개요**
주제 선정 배경 , 구현 목표

02 **시스템 구성 및 개발 환경**
역할 분담 , 활용 장비 , Docker, Database, 전체 아키텍처

03 **핵심 기술 구현**
STT 기반 인터랙션 , CV 기반 견과류 인식 , ROS2 로봇 제어

04 **수행 결과 및 검증**
Pick 안정화 , 예외 처리 , RoboDK 안전영역 , End-to-End 결과 검증

05 **자체 평가 및 개선 방향**
성과 , 한계점 , 향후 보완 계획

주제 및 선정 배경 — 무인 견과 디스펜서

사용자 맞춤 + 협동로봇 + AI(STT/LLM/CV) + SKU 통합



1 헬스 스낵

아몬드 · 캐슈넛 · 피스타치오 · 호두 4 종

2 개인 맞춤형 추천

사용자 직업 · 포만감에 따라 견과류 비율을
동적으로 결정하는 솔루션

3 무인화

키오스크 / 오피스 라운지 / 헬스장 자판기 등
24/7 운영 모델 확산

4 협동로봇 효용 검증

정밀 픽 앤 플레이스 + AI 통합 데모
다품종 SKU 확장 기반 마련



이미지 : 자연드림

기획의도

**음성 한 마디로
맞춤형 견과류 콤보가
사용자 앞에 나옵니다 .**

플로우차트

STT · AI 추천 · Vision · 협동로봇 기반 서비스 흐름



팀 프로젝트 일정

협동2 프로젝트 간트 차트

프로젝트 이름	AI(Computer Vision)기반 협동 로봇 작업 어시스턴트 구현 프로젝트
프로젝트 관리자	A-2
날짜	26년 4월 30일(목) ~ 5월 13일(수)

작업 분류	작업 제목	작업 소유자	시작일	마감일	4/30	5/1	5/2	5/3	5/4	5/5	5/6	5/7	5/8	5/9	5/10	5/11	5/12	5/13
						개방X				개방X								
					목	금	토	일	월	화	수	목	금	토	일	월	화	수
프로젝트 설계 및 환경 구축	요구사항 정의	전원	26년 4월 30일	26년 4월 30일														
	시스템 아키텍처 설계	이승준	26년 4월 30일	26년 4월 30일														
	DB 스키마 설계	이승준, 최진우	26년 4월 30일	26년 4월 30일														
	개발 환경 구축 (Git & Docker)	최진우	26년 4월 30일	26년 4월 30일														
비전 지능 및 데이터 개발	데이터 수집	김기현, 허찬영	26년 4월 30일	26년 4월 30일														
	라벨링	전원	26년 4월 30일	26년 5월 4일														
	YOLOv8 모델 학습	최진우, 최성아	26년 4월 30일	26년 5월 6일														
	모델 검증	최진우, 최성아	26년 4월 30일	26년 5월 6일														
음성 인식 및 데이터 처리	사용자 입력 정의 및 웹UI 구축	이승준	26년 5월 4일	26년 5월 7일														
	STT, TTS 모듈 개발	이승준, 최진우	26년 5월 4일	26년 5월 7일														
	명령어 파싱	최진우	26년 5월 6일	26년 5월 8일														
	Firebase 연동	이승준, 최진우	26년 5월 6일	26년 5월 8일														
	비전 결과와 STT 결합 논리 구조 작성	이승준, 최진우	26년 5월 6일	26년 5월 7일														
로봇 제어 및 좌표 통합	CAD 및 안전영역 설정	허찬영	26년 5월 7일	26년 5월 8일														
	로봇 모션 프로그래밍	김기현, 최진우	26년 5월 6일	26년 5월 8일														
	시스템 통합	전원	26년 5월 8일	26년 5월 11일														
	Docker 배포	최성아	26년 5월 9일	26년 5월 12일														
Challenge	STT 단순 키워드 매칭 방식 해결	최성아	26년 5월 7일	26년 5월 8일														
	컨베이어 벨트 활용	이승준	26년 5월 7일	26년 5월 7일														
	SUPABASE구현	이승준, 최성아	26년 5월 11일	26년 5월 11일														
테스트 및 프로젝트 마무리	시나리오 테스트	전원	26년 5월 11일	26년 5월 12일														
	예외 처리	최진우, 김기현	26년 5월 9일	26년 5월 12일														
	최종 성능 평가	전원	26년 5월 12일	26년 5월 13일														
	문서화 및 완료	허찬영	26년 5월 11일	26년 5월 12일														
	발표	허찬영	26년 5월 13일	26년 5월 13일														발표

훈련생	역할	주도적 참여 영역
허찬영	팀장	• Project Manager • Scheduling & Documentation • Simulation & Safety Engineer • 작업 및 로봇 공정 환경 구성
김기현	팀원	• Mechanical Elements in Robot Motion • AI Vision based Gripper Motion • 예외 처리 관련 알고리즘 구현
이승준	팀원	• Raw Data Collection Management • Frontend & UX/UI • Database & Schema Application • System Architecture • Conveyor (Wiring & Programming)
최성아	팀원	• LangChain • STT Prompt Engineering & HCI • Docker Application • Database Schema Design • AI Vision Modeling & Training
최진우	팀원	• Programming in Robot Motion • Node Architecture • AI Vision Modeling & Training • STT Engineering • Devops / Git / Docker Management

개발 환경

협업 · 개발 · 시뮬레이션 · 데이터 관리 환경 구성

협업 환경

slack

Notion



개발 환경

docker

GitHub



AI/DB 환경

supabase

roboflow

하드웨어 제어 환경

ARDUINO

RoboDK

ROS 2

ubuntu

Docker 기반 개발 환경 - 표준화와 성능의 최적화

컨테이너 기술을 활용한 라이브러리 통합 및 하드웨어 가속 환경 구축

1. 표준화된 개발 환경

- **패키지 통합 :** YOLO, OpenCV, ROS 2 등 필수 라이브러리를 **Docker** 하나로 단일화
- **버전 관리 :** 팀원 간 라이브러리 불일치로 인한 설치 오류를 차단

2. 하드웨어 최적화

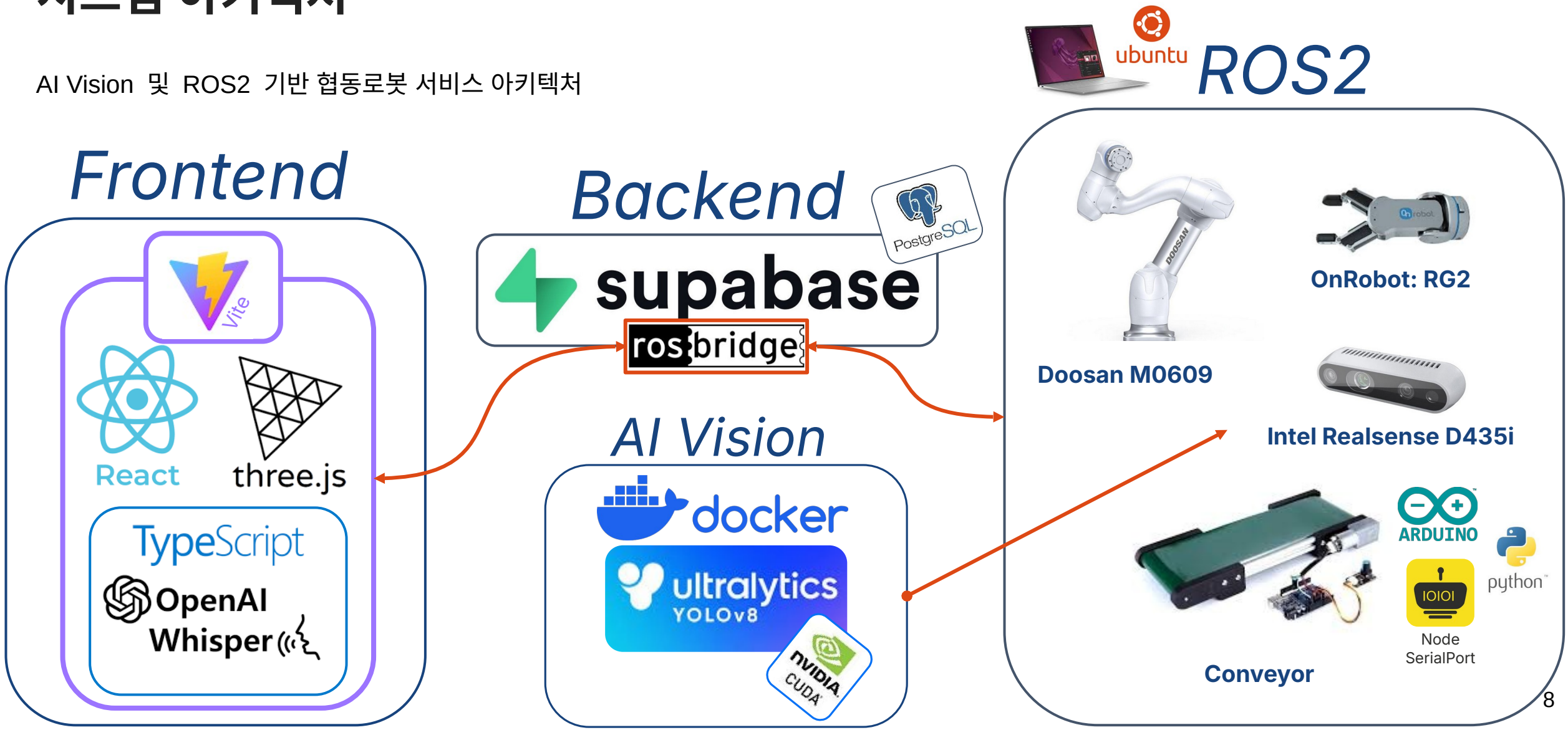
- **GPU 가속 : CUDA** 를 활용해 객체 인식 속도를 극대화하여 실시간 처리가 가능함
- **자동 전환 :** GPU 가 없는 환경에서도 에러 없이 **CPU** 모드로 자동 전환되어 작동

3. 통신 효율 및 편의성

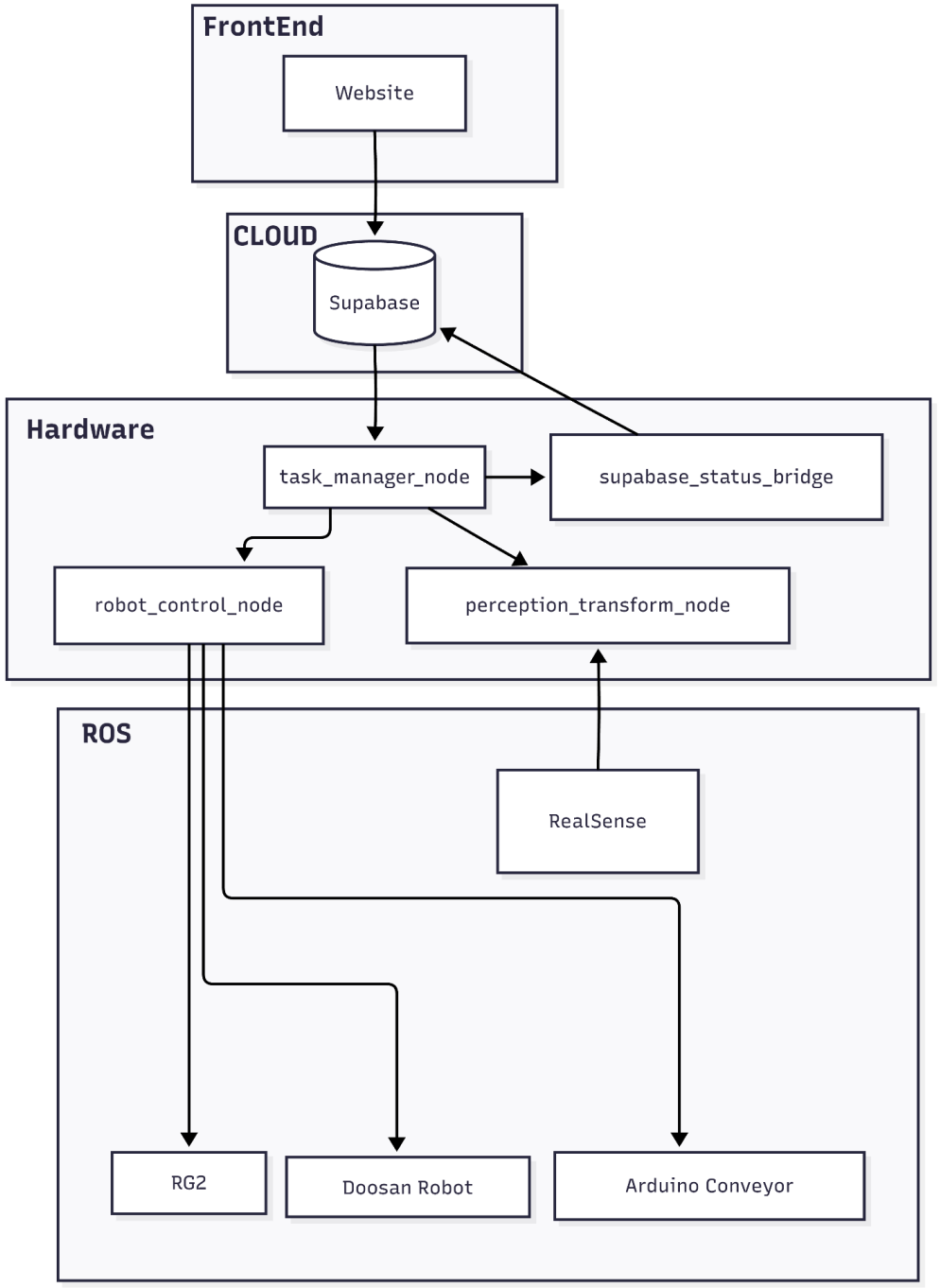
- **지연 최소화 : Host Network** 모드를 사용하여 데이터 전송 지연 문제를 해결
- **간편한 실행 :** 복잡한 과정 없이 명령어 한 줄로 1 분 만에 개발 준비 완료

시스템 아키텍처

AI Vision 및 ROS2 기반 협동로봇 서비스 아키텍처



노드 아키텍처


FrontEnd
Website
CLOUD
Supabase
Hardware
task_manager_node
supabase_status_bridge
robot_control_node
perception_transform_node
ROS
RealSense
RG2
Doosan Robot
Arduino Conveyor

SUPABASE 클라우드 DB - 견과류 재고 관리 및 오류 로그 기록

문제 원인 파악 및 재고 관리 시스템을 통한 운영 효율성 극대화

재고 관리

사람이 직접 세지 않아도 데이터베이스가 실시간 재고를 자동으로 계산하여 오차 없는 관리 환경을 제공합니다 .

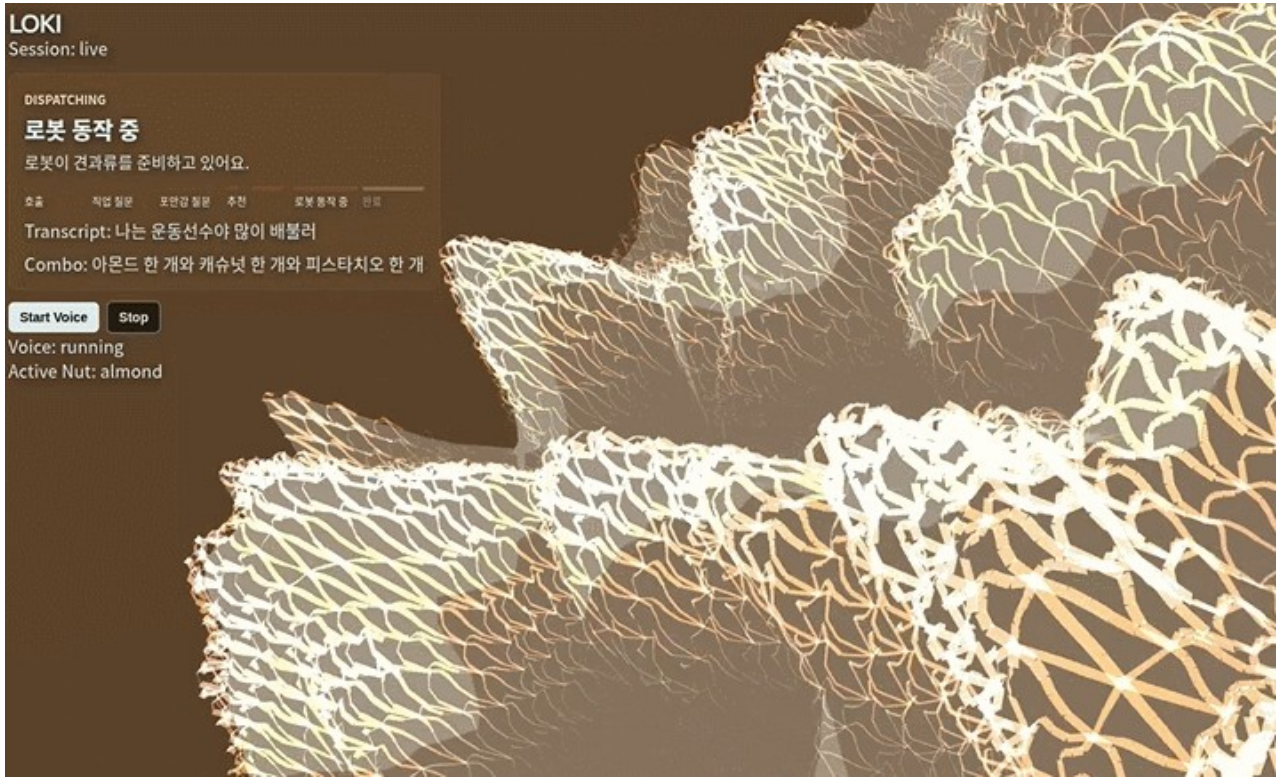
문제 원인 파악

작업 중 발생한 특이사항 및 로그를 실시간으로 기록하여 로봇의 동작 개선 및 유지보수 자료로 활용합니다 .

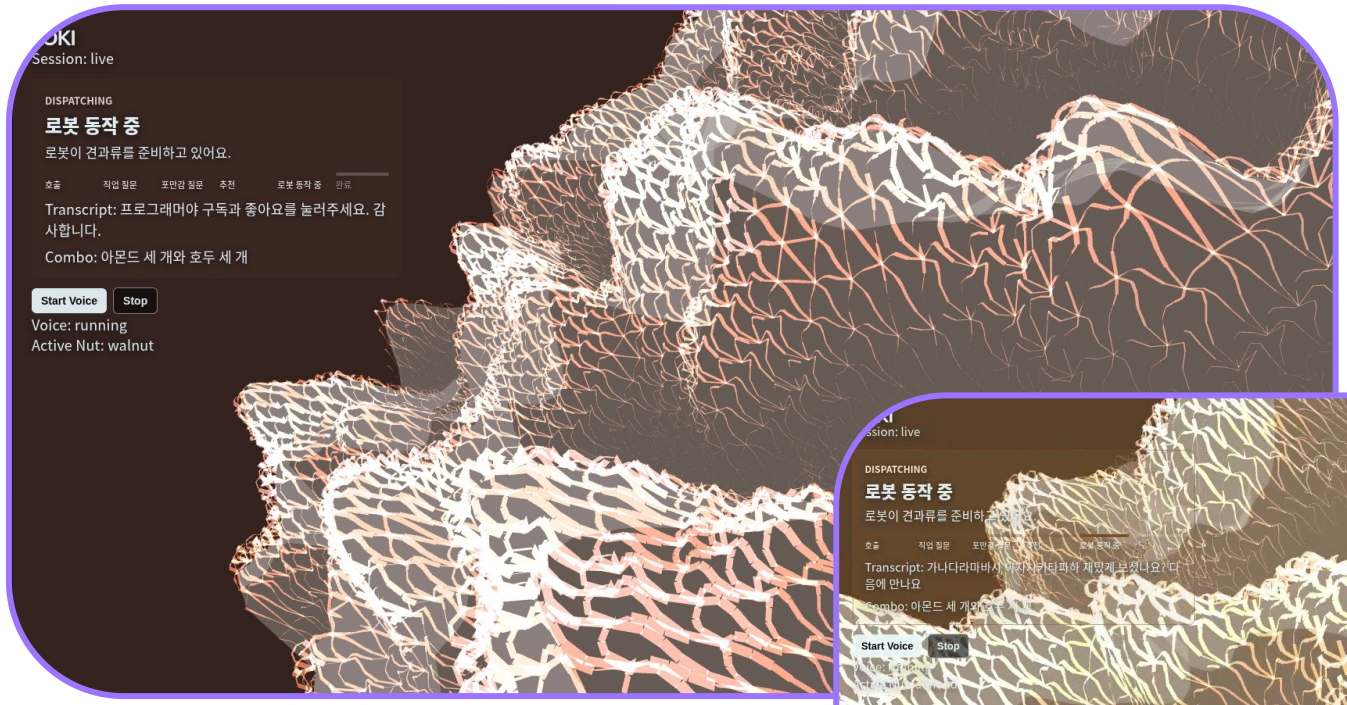
테이블 명	주요 관리 항목	활용 목적
재고 현황	견과류 4 종 수량	실시간 재고 모니터링
변동 이력	입출고 수량 , 변동 사유	재고 변화의 흐름 파악 및 투명한 이력 관리
예외 로그	작업 상태 , 위치 정보 , 오류 메시지	예외 발생 원인 분석 및 개선

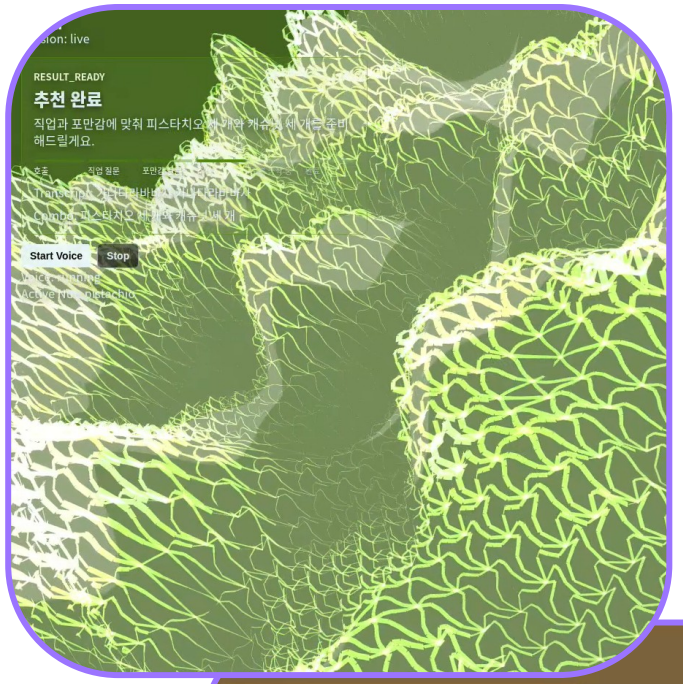
노드 아키텍처 - 웹 UI

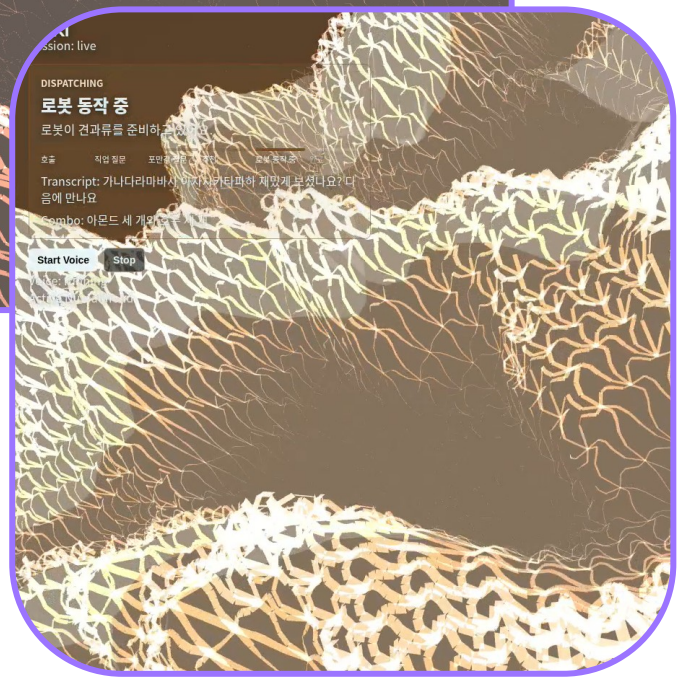

LOKI
Session: live
대기 중
호출어를 말해주세요.
Start Voice
Stop
Voice: running
Active Nut: none
LibreOffice Impress


LOKI
Session: live
DISPATCHING
로봇 동작 중
로봇이 견과류를 준비하고 있어요.
Transcript: 나는 운동선수야 많이 배불러
Combo: 아몬드 한 개와 캐슈넛 한 개와 피스타치오 한 개
Start Voice
Stop
Voice: running
Active Nut: almond

노드 아키텍처 - 웹 UI(견과 종류별 UI)









Supabase 테이블 용도

상태와 이력 관리

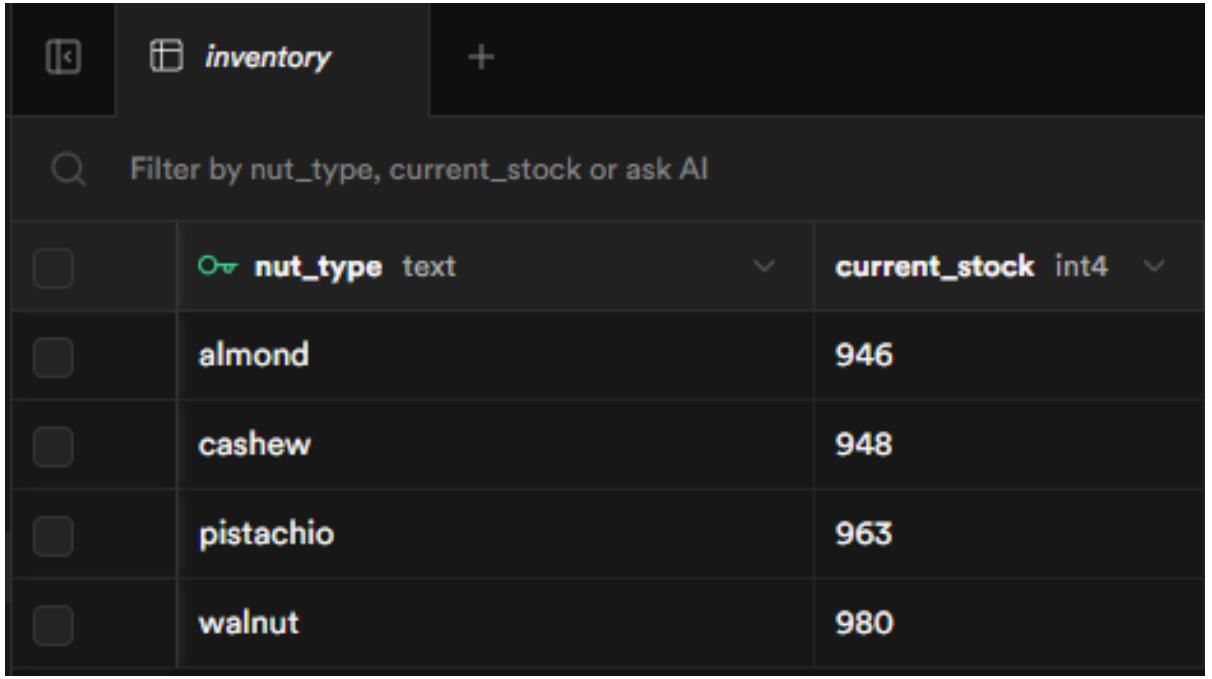
테이블	한 줄 용도
robot_session	웹 ↔ ROS 실시간 세션 상태 단일 소스
inventory	견과류별 현재 재고 (스냅샷)
inventory_logs	모든 재고 변동의 시계열 원장
exception_logs	task 실패/이상 상태 기록

Robot Session

UI 와 ROS 를 연결하는 상태 채널

id	항상 'current'
display_state	UI 단계 식별자 (idle / listening / confirming / …)
question	챗봇이 사용자에게 던진 마지막 질문
transcript	사용자의 마지막 음성 → 텍스트 결과
categories	선택된 카테고리 리스트
combo	추천 견과류 콤보 (배열)
theme	UI 색/테마 정보
intensity	강도 (mild/medium/strong)
combo_text	콤보의 사람이 읽는 텍스트
confirm_message	확인 단계 메시지
error	사용자 흐름 에러 메시지
success	주문 확정 여부 (ROS 는 true 만 읽음)
request_id	주문 dedup 용 UUID — ROS provider 가 이 값으로 새 주문 판별
robot_state	idle / picking / placing / done 등
robot_target_class	현재 픽 중인 견과류 종류 (UI 색상 갱신용)
updated_at	BEFORE UPDATE 트리거 touch_updated_at 이 매번 now() 박음

재고 Tracking


inventory
Filter by nut_type, current_stock or ask AI
nut_type text
current_stock int4
almond
946
cashew
948
pistachio
963
walnut
980

재고 변동 로그

id uuid	nut_type text	change_amount i...	reason text	created_at timestamptz
006af3dc-0cb4-49e8-ad8e-f6f314a65796	cashew	-1	primary_pick_success	2026-05-12 06:13:27.725507+00
051d8461-385b-4162-a798-7ad7c99cbd1f	walnut	-1	primary_pick_success	2026-05-12 01:27:54.753875+0(
05d986d4-3c74-4eac-a981-33a0dc850757	cashew	-1	primary_pick_success	2026-05-12 06:17:46.394384+0
0675978a-df6f-4818-9247-3ed1e94a07e3	pistachio	-1	primary_pick_success	2026-05-12 01:15:28.99521+00
07a22082-2f86-41a9-be2a-306217ee5b33	cashew	-1	primary_pick_success	2026-05-12 01:41:41.489799+00
085046f0-d44c-4e12-a72a-b45515b3b0d7	cashew	-1	primary_pick_success	2026-05-12 09:41:09.424062+0
08e7ea24-bece-45b2-acc8-1d472d9d659(	cashew	-1	primary_pick_success	2026-05-11 08:31:37.41754+00
0cd71cfd-7217-4dcd-b647-caf12520c000	cashew	-1	verify1_pick_success	2026-05-11 07:31:49.505374+0(
0de4f803-a042-4f26-ad32-c7da5387244a	almond	-1	verify1_pick_success	2026-05-12 02:43:19.686094+0
0e19b43f-7cf4-4436-be0d-9dd9653cb693	cashew	-1	primary_pick_success	2026-05-12 09:41:35.168851+00
1037a563-3389-4f55-8665-ba95fcd76392	cashew	-1	primary_pick_success	2026-05-12 09:36:44.49687+0(
1180f373-b1b4-4b0b-ab72-a9ef78e685eb	almond	-1	primary_pick_success	2026-05-12 05:29:29.97306+0(
11b62a44-692f-47ff-8ede-2a4756710008	pistachio	-1	primary_pick_success	2026-05-12 01:12:20.425361+00
120415cf-23b4-419e-9352-779375a2ff60	cashew	-1	primary_pick_success	2026-05-12 09:48:50.101444+0(

LangChain & UI/UX 연동 — 지능형 인터랙션 구현

LLM 기반의 상황 인지 추천과 사용자 중심의 직관적인 피드백 시스템

똑똑한 맞춤 추천 (GPT-4o)

- **직업 맞춤 :** 개발자 (눈 건강), 운동선수 (근육) 등 직업별 최적 견과류 매칭
- **포만감 조절 :** 대화 맥락에서 사용자의 배고픔 정도를 파악하여 제공량 가이드
- **의도 추론 :** 키워드 매칭을 넘어 사용자 발화의 전체 의도를 LLM 으로 분석

UI/UX 포인트

- **실시간 상태 공유 :** 로봇의 현재 동작 단계를 화면에 즉각적으로 시각화
- **적응형 테마 :** 추천된 견과류의 종류에 따라 인터페이스 색상이 유연하게 변경

예외 상황 핸들링

- **자동 추천 :** 의도 파악이 불가능할 경우 , 랜덤 견과류 2 종 서빙
- **기본 수량 :** 수량 파악이 불가능할 경우 , 기본 3 개 제공

UI 및 TTS 동시 전달

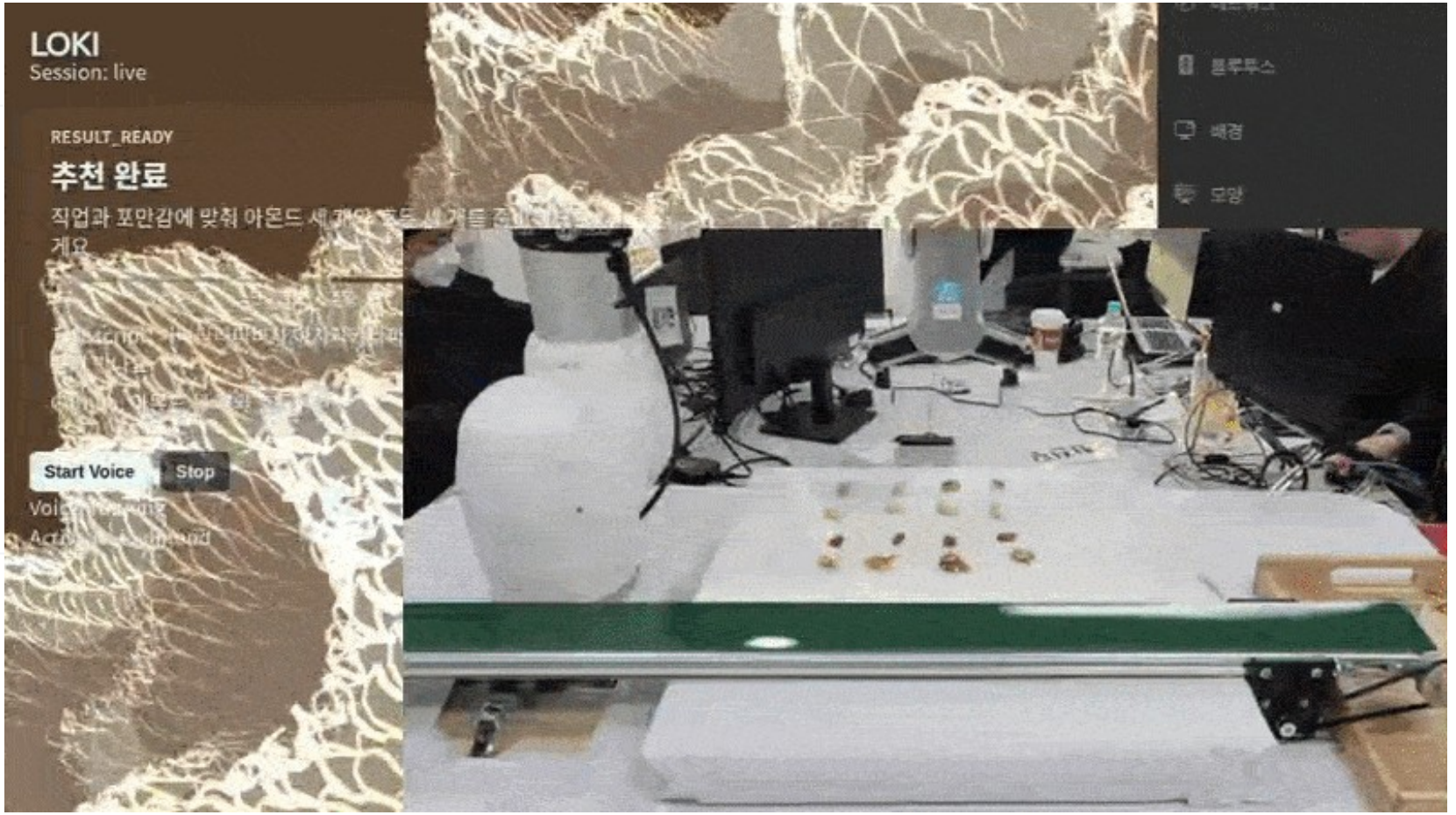
- **최소한의 텍스트 :** 핵심 정보 위주의 간결한 텍스트 배치
- **음성 보조 (TTS):** 추천 사유와 상세 설명은 친근한 음성으로 전달하여 몰입감 증대

LangChain & UI/UX 연동 — 지능형 인터랙션 구현

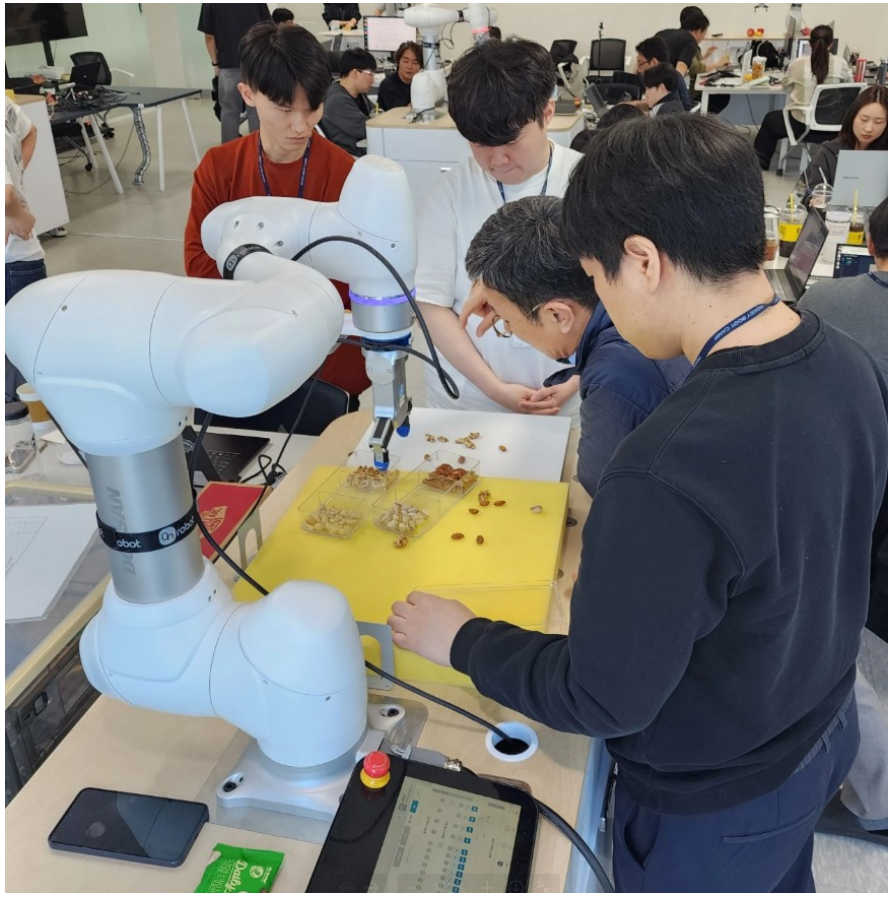
예외 상황 핸들링 영상

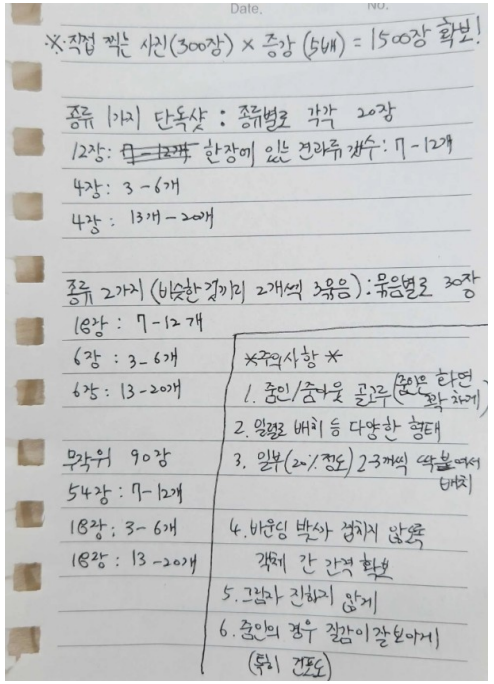
예외 상황 핸들링

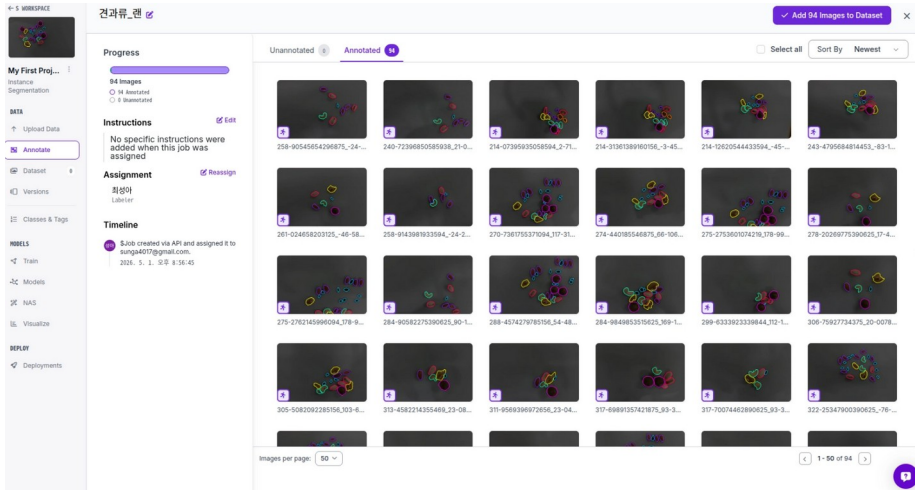
- **자동 추천 :** 의도 파악이 불가능할 경우 , 랜덤 견과류 2 종 서빙
- **기본 수량 :** 수량 파악이 불가능할 경우 , 기본 3 개 제공



이미지 학습




※작업 꺾는 사진(300장) × 증강 (5배) = 1500장 확보!
종류 1가지 단독샷 : 종류별로 각각 20장
12장: 한장에 있는 견과류 개수: 7-12개
4장: 3-6개
4장: 13개-20개
종류 2가지 (비슷한 것끼리 2개씩 3묶음): 묶음별로 30장
18장: 7-12개
6장: 3-6개
6장: 13-20개
무작위 90장
54장: 7-12개
18장: 3-6개
18장: 13-20개
※주의사항※
1. 종인/종하을 골고루(많은 한면 확 차게)
2. 일렬로 배치 등 다양한 형태
3. 일부(2~3정도) 23개씩 딱붙여서 배치
4. 바운딩 박스 겹치지 않을 객체 간 간격 확보
5. 그림자 진하지 않게
6. 종인의 경우 잘감이 잘보이게 (튀김 견포)


견과류_랜
Add 94 Images to Dataset
Progress
Instructions
No specific instructions were added when this job was assigned
Assignment
Timeline
Unannotated
Annotated
Select all
Sort By
Newest

1 차 — YOLOv8n (일반 바운딩 박스)

학습 설정 요약

데이터 : Roboflow 23 개 데이터셋 통합 · 396 장 / 2,992 박스 · 8:1:1 분할

클래스 : almond 747 / cashew 756 / pistachio 750 / walnut 739 (균등)

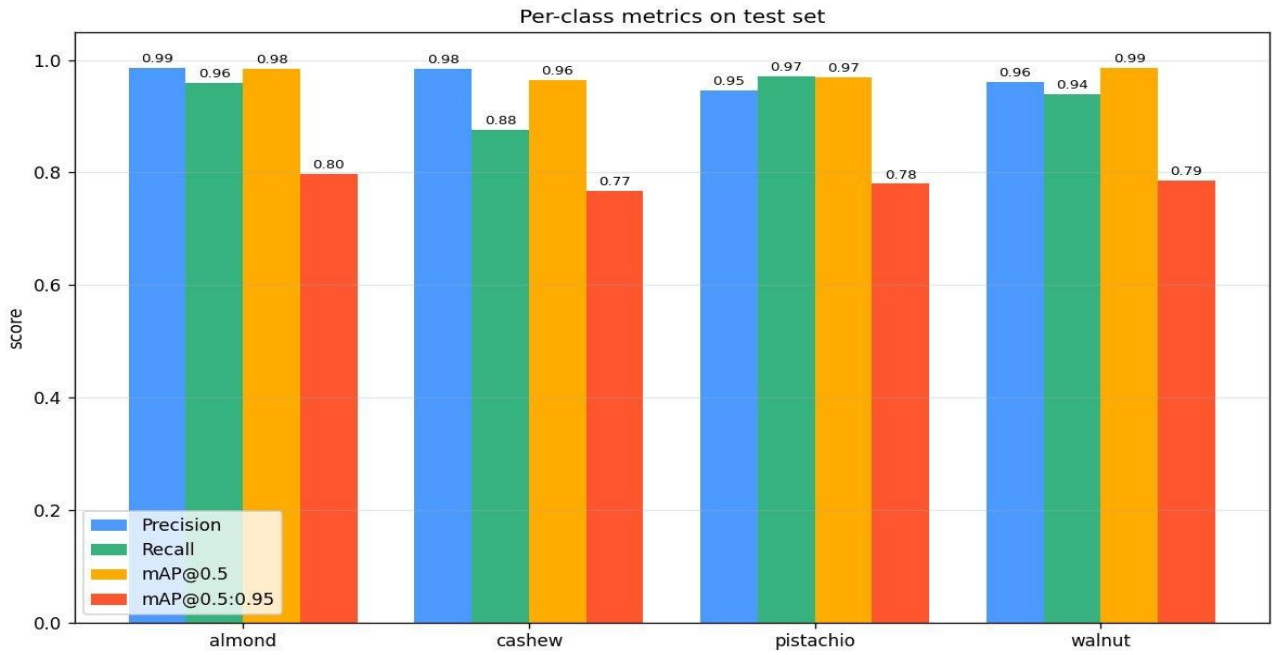
학습 : 200 회 예정 → 157 회 자동 종료 · RTX 5060 · 6 분 36 초

증강 : mosaic, HSV 변환 , 회전 / 이동 / 확대

결과 (test set)

0.97	0.94	0.985	0.79
Precision	Recall	mAP@0.5	mAP@0.5:0.95

클래스별 성능 (테스트셋)



폐기 사유 4 가지

1 회전 각도 X

박스가 똑바로 배치됨 → PCA 별도 계산 필요

2 박스 겹침

군집한 견과류 사진에서 객체끼리 묶임

3 cashew 12% 누락

긴 객체가 NMS 에서 사라짐

4 다운스트림 부담

회전각 / 중심 / 자세 후처리 추가

1 차 — YOLOv8n (일반 바운딩 박스)

Cashew 을 착각 !
- Pistachio
- Walnut
- Background (혹은 , Empty…)

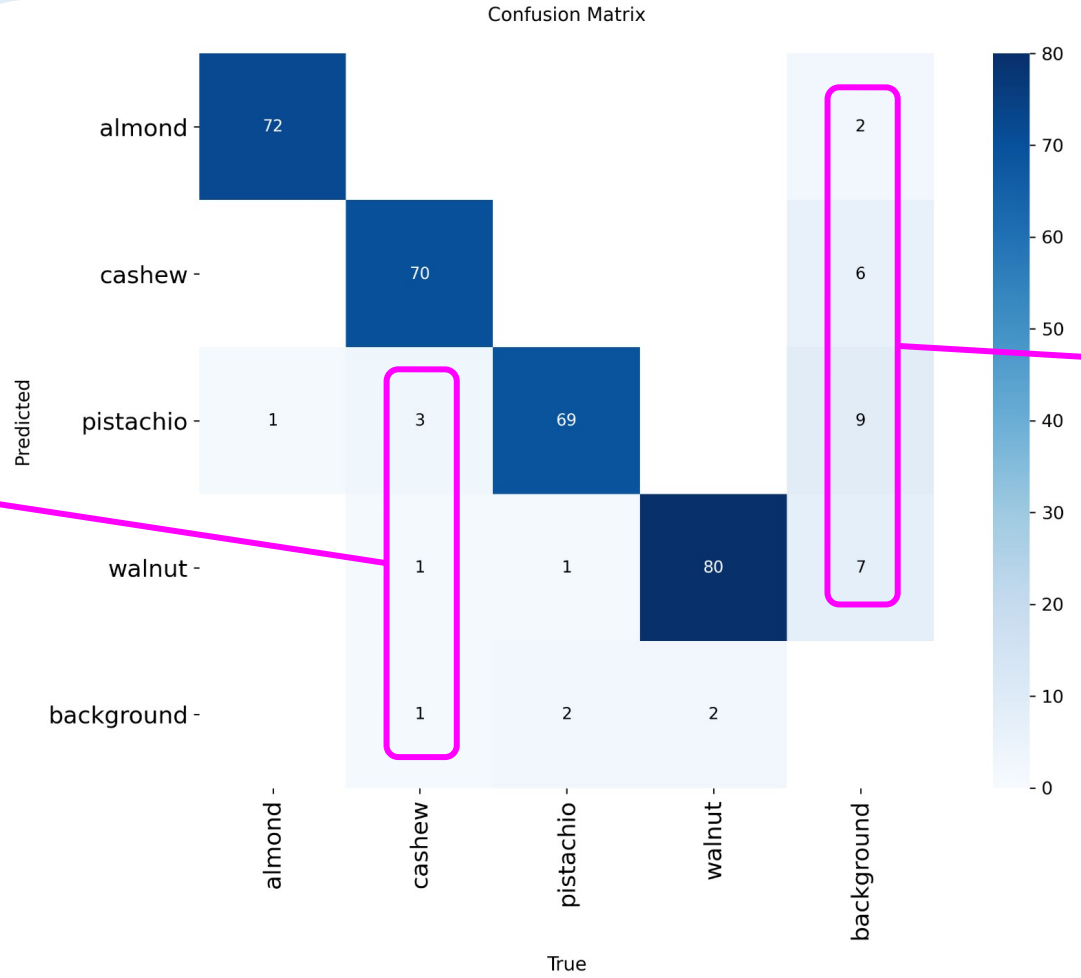
→ ***캐슈넛의 데이터를 보강하되…***
→ ***피스타치오랑 호두를 섞어서…***
→ ***헷갈려하는 정도를 맞춰서…***
→ ***하지만 , 캐슈넛의 갯수가 월등히 많게 !***

예시) Cashew:Pistachio:Walnut = 8:2:1

OR

Cashew:Pistachio = 9:3

… 이런식으로 50 장의 사진 추가 !



- 한 개의 견과류만 찍힌 사진이 있지만 , 어떤 견과류도 없는 배경들이 없이 훈련한 결과
- 어떠한 경우가 빈 (Empty) Background 인지 알지 못하나… ?
- 그래서 , 4 가지 견과류 종류 중 , 어떠한 견과류에도 해당되지 않다고 판단할 때 , ‘Background’ 라고 생각하나… ?

→ ***아무 견과류도 없는 ‘Empty Background’ 사진 30 장 추가 !***

First Model Training: Validation Confusion Matrix

2 차 — YOLOv8n-OBB (회전 박스 , 최종 채택)

★ FINAL CHOICE

설계 변경 포인트

1 OBB 헤드 교체

yolov8n-obb.pt 미리학습 가중치 사용 . nano 본체에 회전 박스 출력 헤드만 추가 .

2 2 단계 학습 도입

1 단계 80 회 + (mosaic/copy_paste 강하게) → 2 단계 30 회 (mosaic 끄고 깨끗한 사진으로 fine-tune).

3 라벨 형식

class x1 y1 x2 y2 x3 y3 x4 y4 (꼭짓점 4 개) — 회전 각도가 자연스럽게 포함 .

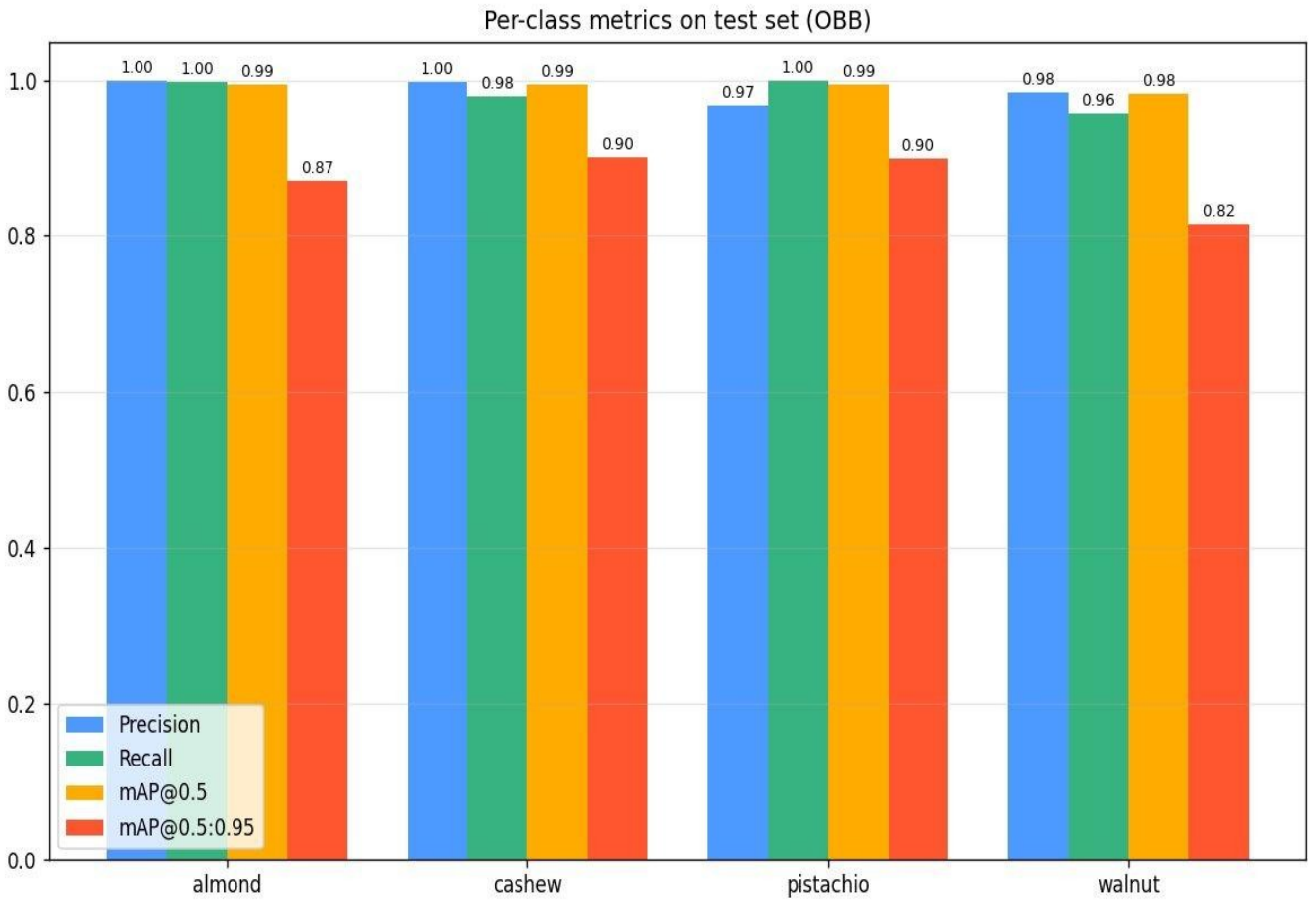
4 conf/iou 스윕

conf 0.10~0.60 × iou {0.5, 0.6, 0.7} 모두 시도 . conf=0.40, iou=0.5 에서 F1 0.9931 최고 .

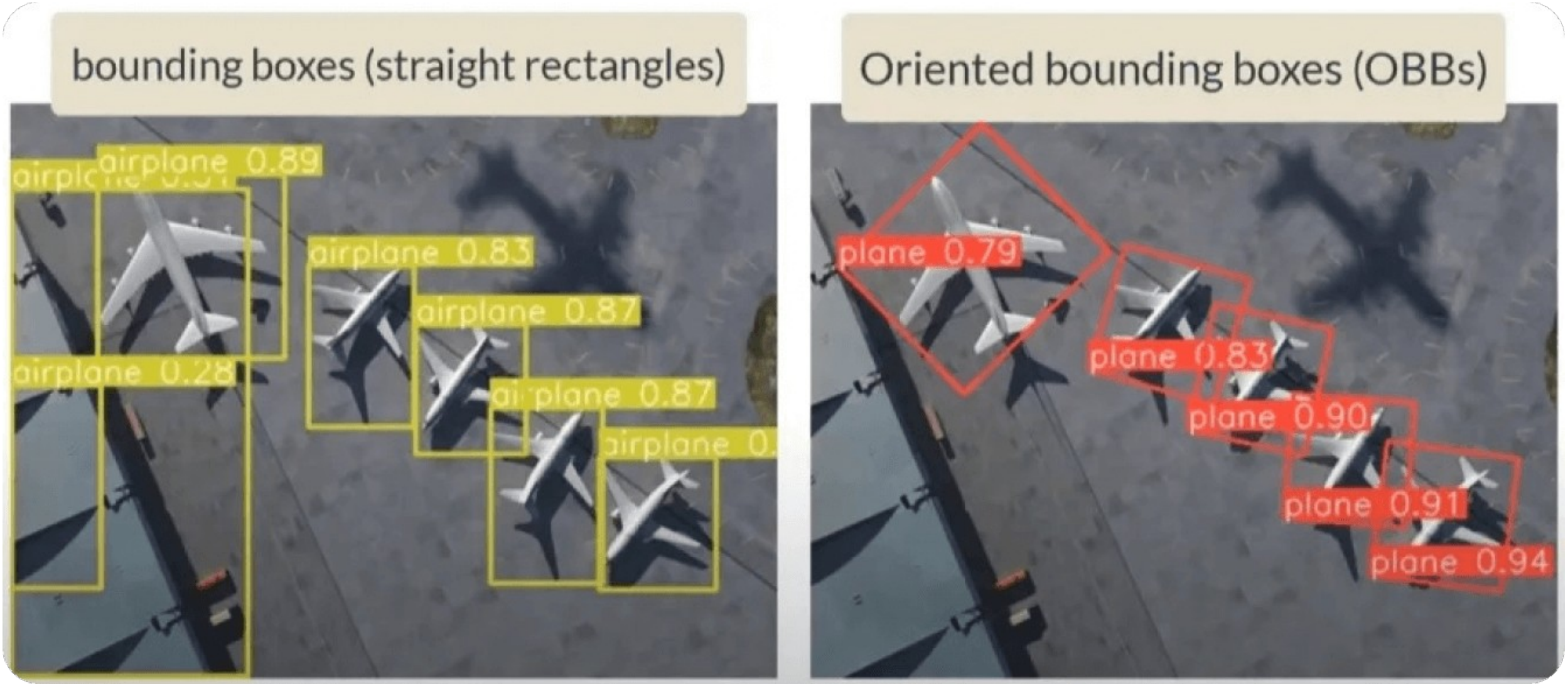
무엇이 달라졌나

12% → 2%	+0.08	−100 줄	8–10 분
cashew 놓침 비율	mAP@0.5:0.95 평균	PCA 후처리 코드	학습 시간

테스트셋 클래스별 (conf=0.40 / iou=0.5)

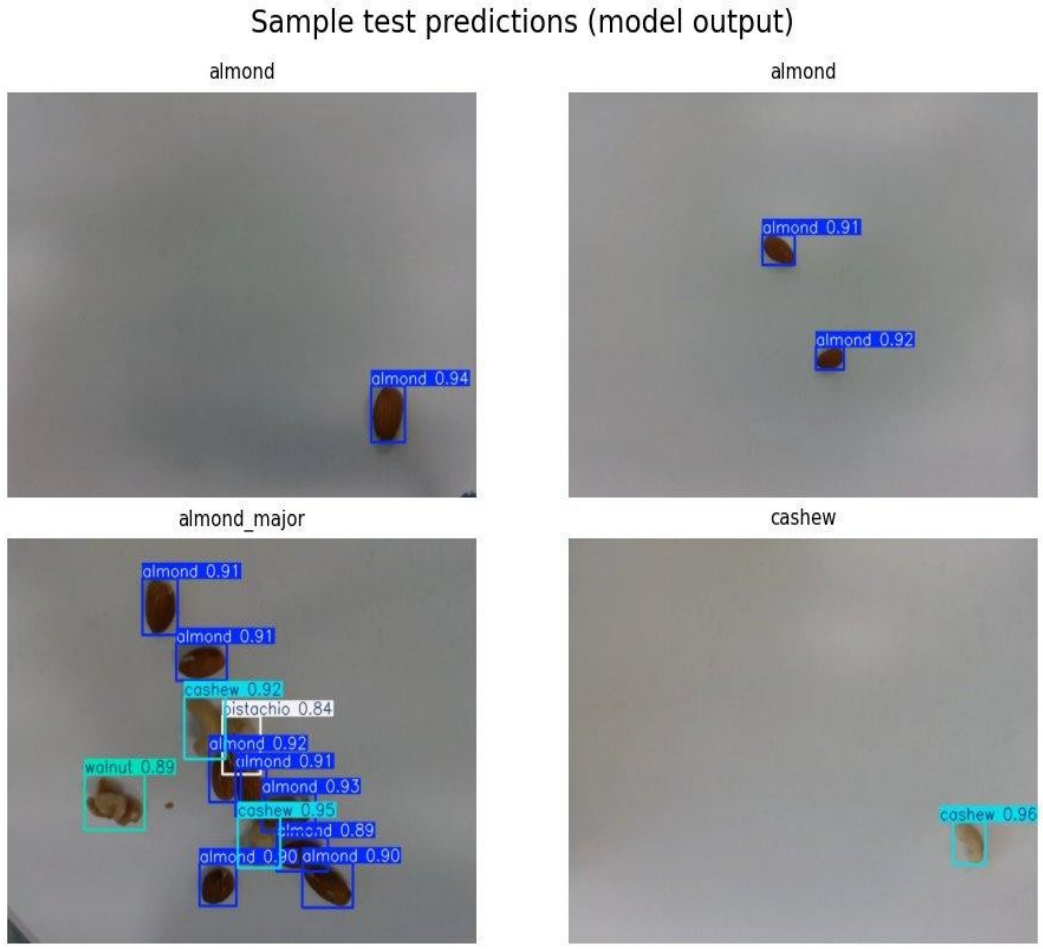


Bounding Box VS Oriented Bounding Box 샘플 비교

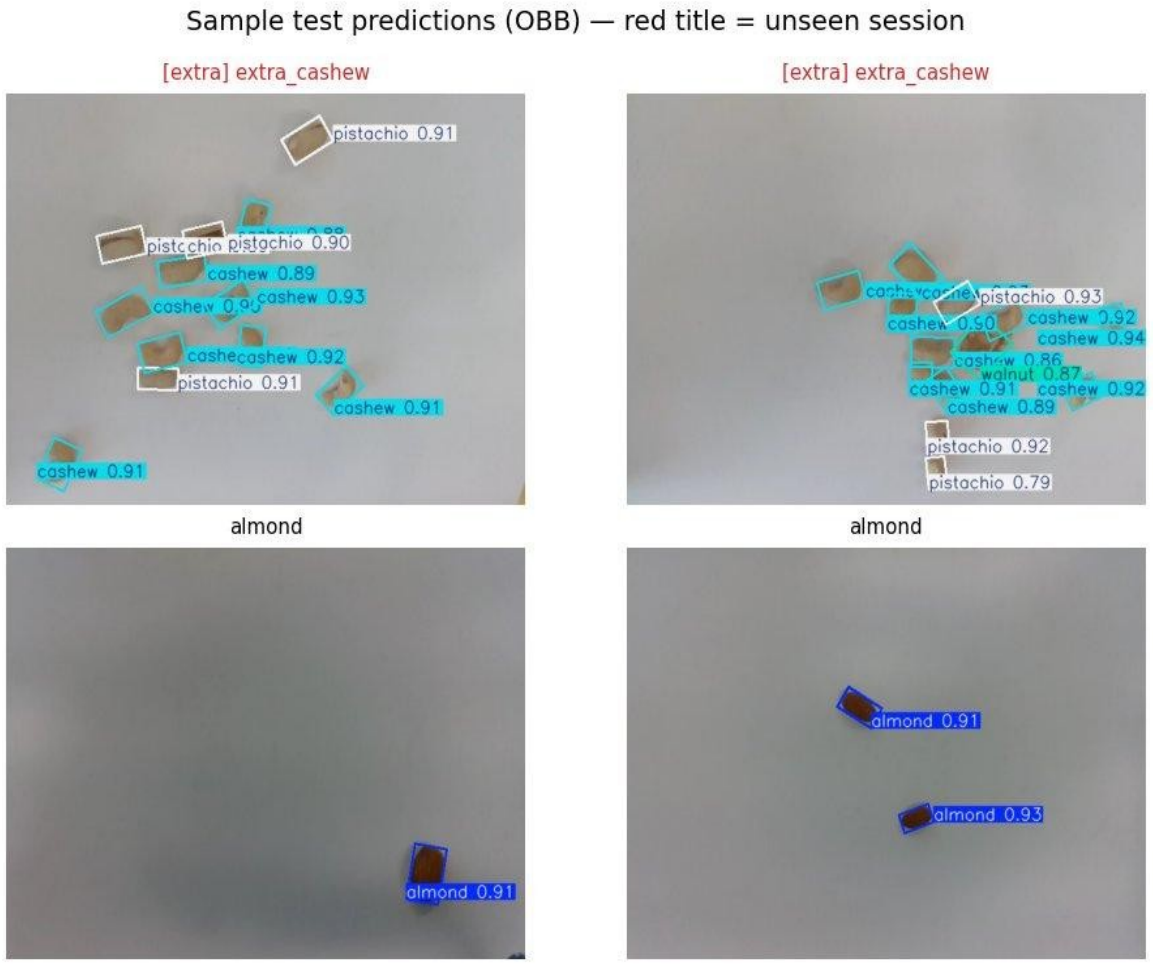


이미지 : Ultralytics

Bounding Box VS Oriented Bounding Box 샘플 비교



1 차 일반 박스



2 차 nano ★

3 차 — YOLOv8s-OBB (큰 회전 박스)

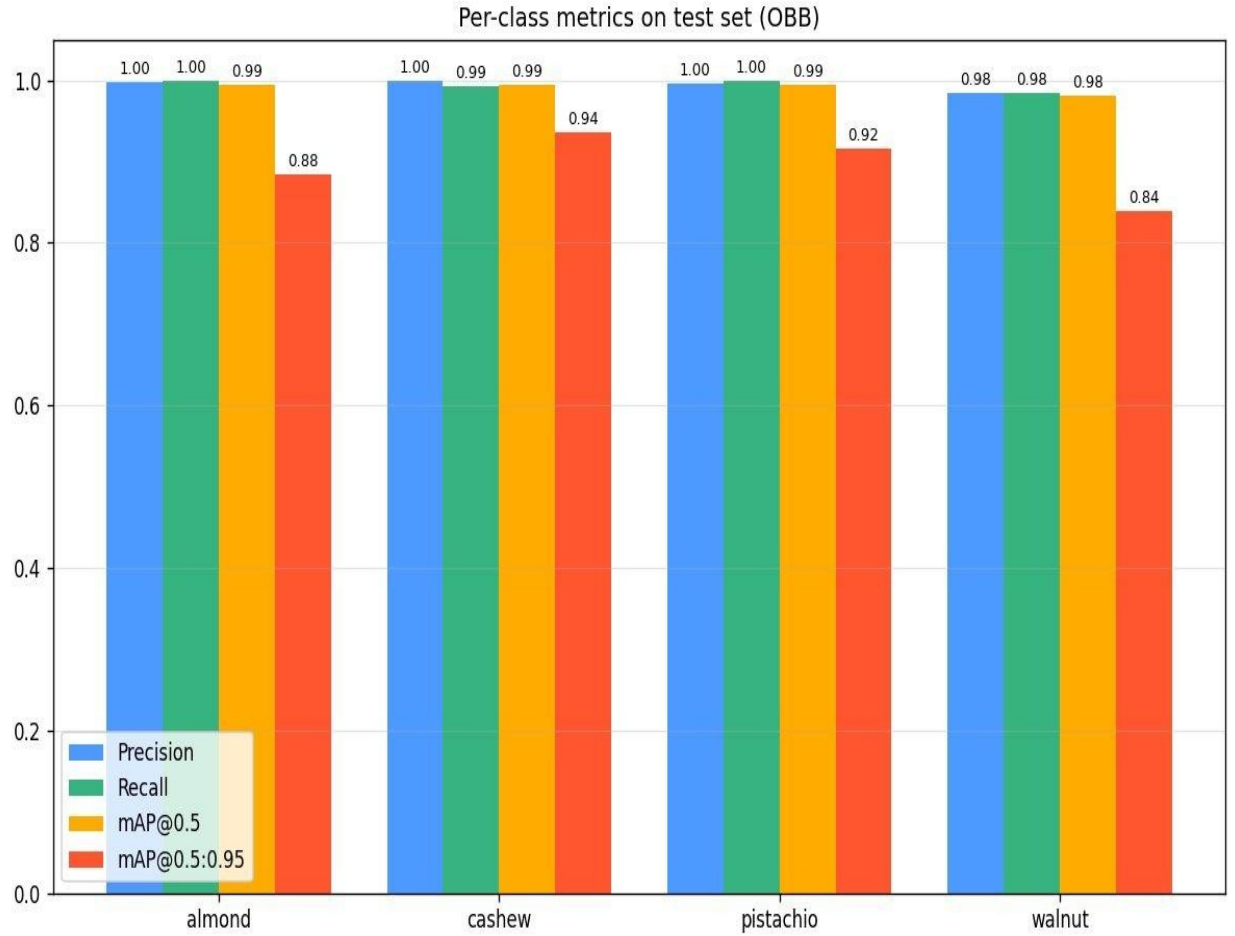
왜 큰 모델을 시도했나

- nano 에서 walnut(호두) 엄격 정확도 0.82 가 다른 클래스 (0.87~0.90) 보다 낮은 약점
- **표현력 ↑** → 호두 껍질의 거친 음영을 더 잘 잡을까 하는 기대
- 1 차 보고서에서도 "yolov8s 로 키워보자 " 가 다음 단계 권장

학습 설정

- 데이터 /2 단계 구조 동일 · **본체만 nano → small 교체**
- 1 단계 약 80 회 + 2 단계 25 회
- 신뢰도 스윕 : conf=0.35~0.45, iou=0.6 에서 F1 0.9950 평탄 구간

테스트셋 클래스별 (conf=0.40 / iou=0.6)



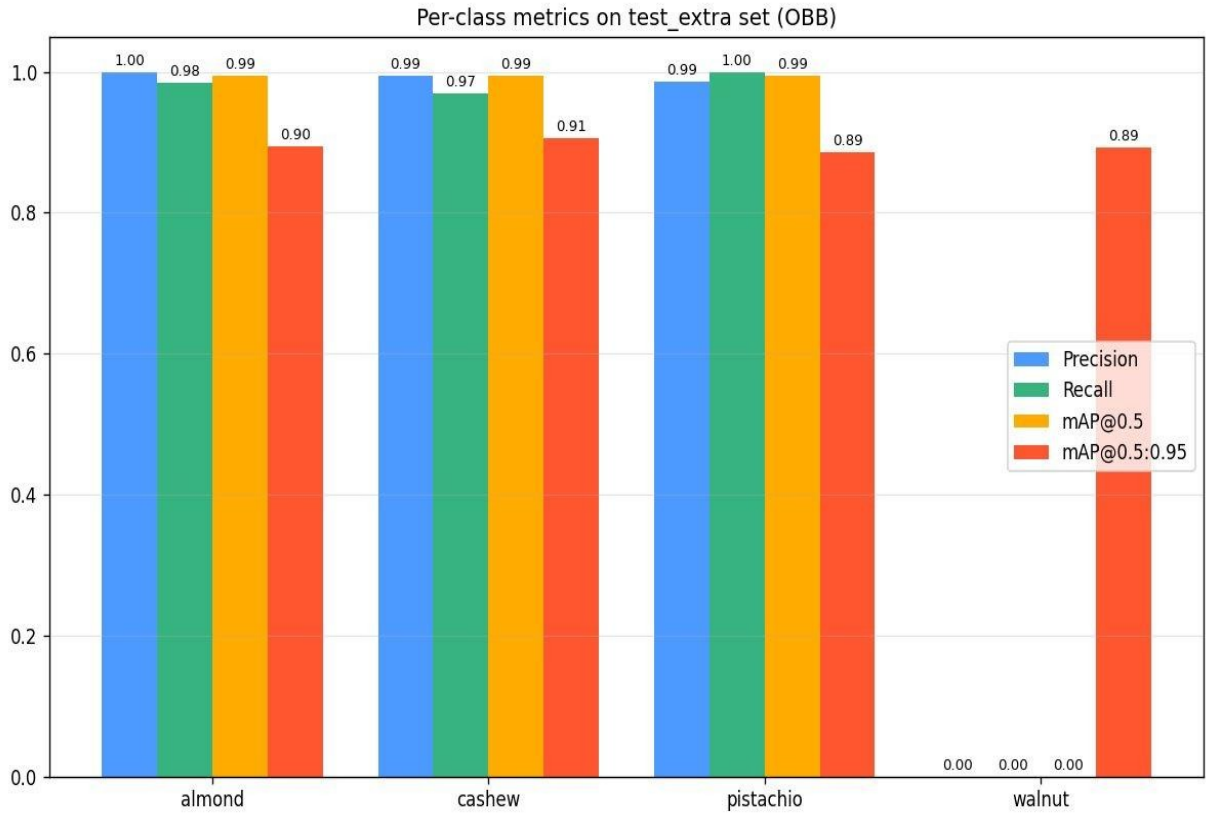
3 차의 함정 - 새 환경에서 호두 0 건 검출

0.00 walnut Precision / Recall / mAP@0.5
새 환경 사진 (test_extra) · small 모델

vs

0.98 같은 모델 , 같은 환경 테스트셋에선 정상

test_extra 클래스별 — 호두 막대 3 개 모두 바닥



무엇이 일어났나 — 추정 원인

- test_extra 는 조명 · 테이블 무늬 · 카메라 각도가 다른 새 촬영 . 다른 3 개 클래스는 잘 잡았지만 **호두에서 크게 실패**
- **추정 :** 큰 모델 (small) 의 표현력이 학습 사진 속 호두 껍질 무늬에만 너무 익숙해짐
  → 새 환경의 다른 광택을 호두로 알아보지 못함 (overfitting 의 일종)
- nano 는 학습 환경 호두 점수 0.82(small 의 0.84 보다 낮음)
  → **학습 데이터에 덜 들러붙은 표현이라는 신호 — 새 환경 일반화에는 오히려 유리**

최종 선택 기준

평가 항목	1 차 일반박스	2 차 nano ★	3 차 small
분류 정확도 (테스트 mAP@0.5)	0.985	0.99 ★	0.99
박스 정밀도 (mAP@0.5:0.95)	0.79	0.87	0.90
회전 각도 직접 출력	×	✓ ★	✓
추론 속도	7ms	8ms ★	30ms (3.75 배 ↓)
모델 크기	6MB	6MB ★	22MB
새 환경 일반화	측정 안 함	(상대) 안정	호두 0.00 붕괴 ★
다운스트림 코드	회전각 별도	모델이 출력 ★	동일
학습 비용	약 30 분	약 40 분	약 4 시간 30 분 ★
선택	폐기	채택 ★	폐기

세 모델 핵심 지표 한눈에 비교

지표	1 차 일반박스 yolov8n	2 차 회전박스 ★ yolov8n-obb (nano)	3 차 회전박스 yolov8s-obb (small)
모델 크기 (파라미터)	3.2M	3.1M	11.4M
학습 방식	1 단계 200 회	2 단계 (80+30 회)	2 단계 (80+25 회)
검증셋 mAP@0.5	0.993	0.99	0.99
검증셋 mAP@0.5:0.95	0.87	0.88 ~ 0.91	0.92 ~ 0.94
테스트셋 mAP@0.5	0.985	0.99	0.99
테스트셋 mAP@0.5:0.95	0.79	0.82 ~ 0.90	0.84 ~ 0.94
신뢰도 튜닝 후 F1	(안 함)	0.9931 ★	0.9950
새 환경 테스트 (test_extra)	(안 함)	(안 함)	walnut 모두 0.00 ★
추론 속도 (RTX 5060)	약 7ms	약 8ms ★	약 30ms
회전 각도 출력	× (PCA 별도)	✓ 자동 ★	✓ 자동

핵심 관전 포인트

small 이 nano 보다 +0.02~0.04 점수가 높지만, 그건 2.3 배 큰 모델의 결과 — 그리고 test_extra 에서 호두 0 건. 점수 우위는 학습 환경 안에서만 유효.

운영 설정 — cobot_object_detection

cobot_object_detection/config/object_detection.yaml

```yaml
weights:        cobot_OD_obb_nano/
                train_phase2_20260504_173049/weights/best.pt
conf_threshold: 0.75   # 스윕 최적값 0.40 보다 보수적 — FP 차단 우선
iou_threshold:  0.50
imgsz:          800    # 학습 imgsz 640 → 운영 800
                       (RealSense 컬러 해상도)
device:         cuda:0 # 백업: cpu (mock 모드)
```

운영 핵심 결정

WHY 0.75

conf 0.75 (보수적)

스윕 최적값은 0.40 이지만 운영에서는 오검출 (FP) 차단 우선 . 그리퍼가 잘못 잡으면 사이클 중단이 더 손해 .

WHY 800

imgsz 800 (학습 640)

RealSense D435 컬러 1280×720 입력 . 학습은 640 으로 빠르게 , 운영은 800 으로 해상도 양보 .

FALLBACK

cpu mock 모드

GPU 미장착 시 cpu 로 강제 — best.pt 는 그대로 둔 채 device 만 교체 .

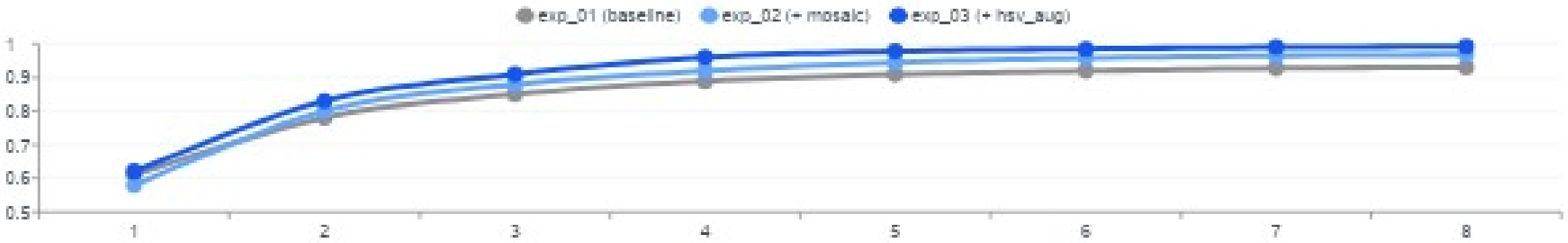
최종 **YOLOv8n-OBB (nano) · conf=0.75 / iou=0.50 / imgsz=800** · 8ms 추론 , 30 초 픽업 사이클 안에서 5 회 검출 여유

모델 학습 결과 — 3 회 실험을 통한 정확도 향상

exp_01 baseline → exp_02 augmentation 추가 → exp_03 최적화로 mAP 0.992 달성

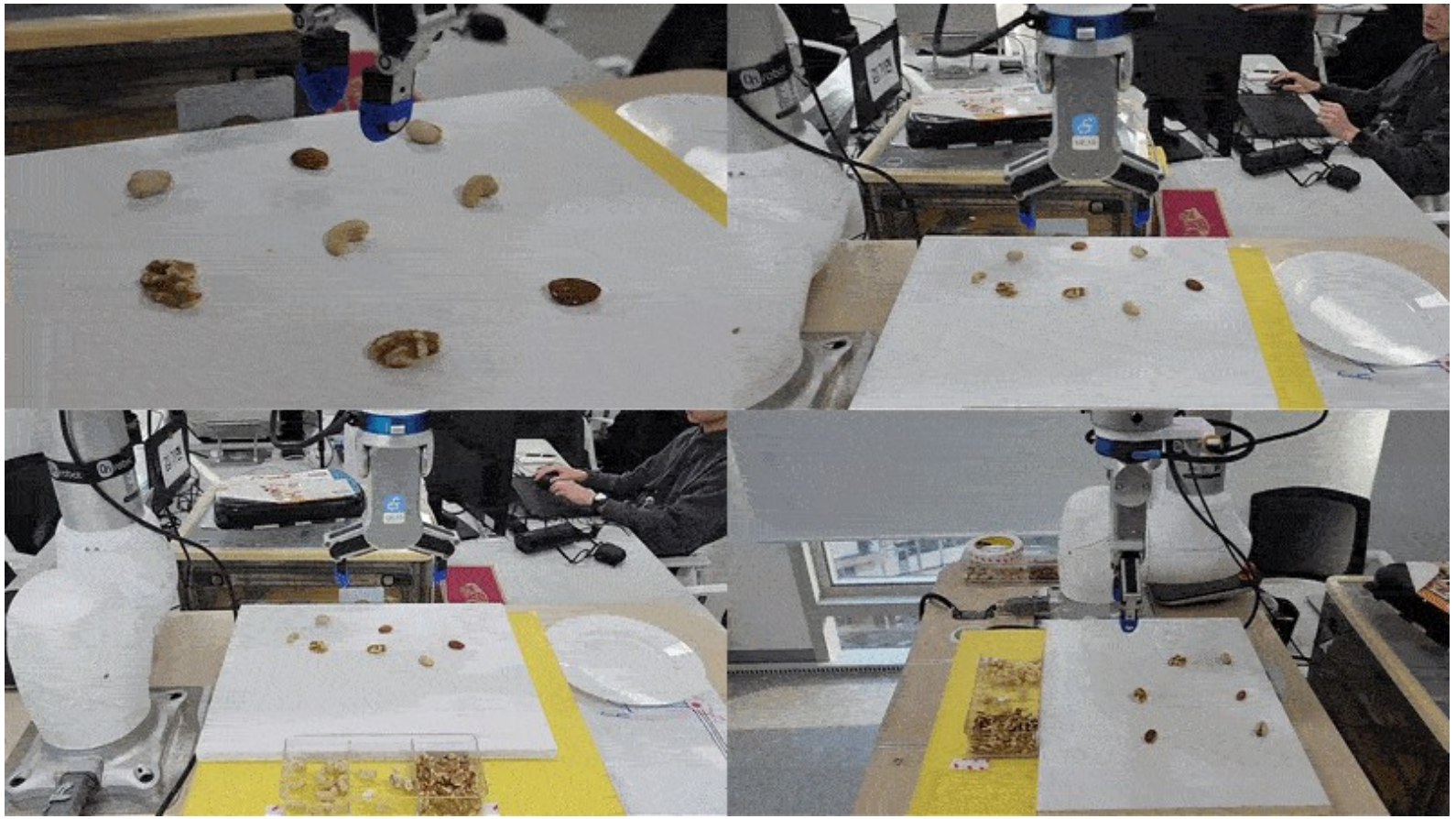
0.992	0.847	~15ms	4/4
test mAP@0.5	test mAP@0.5:0.95	inference 시간	클래스 모두 ≥ 0.98
NFR ≥0.97 통과	엄격 기준	NFR ≤50ms 통과	균등 성능

실험별 mAP@0.5 추이 (epoch 진행에 따른 val mAP)

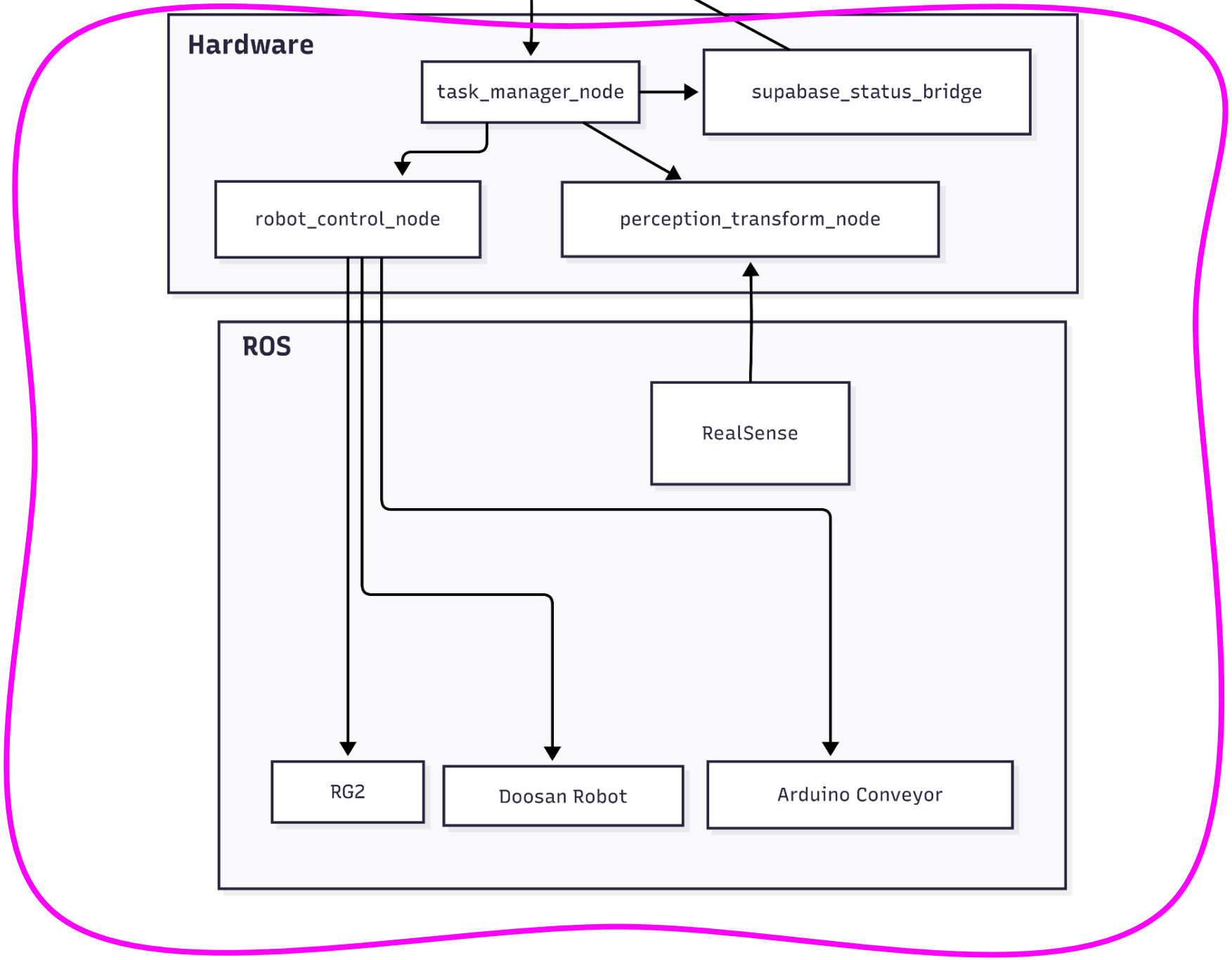


모델 학습 결과

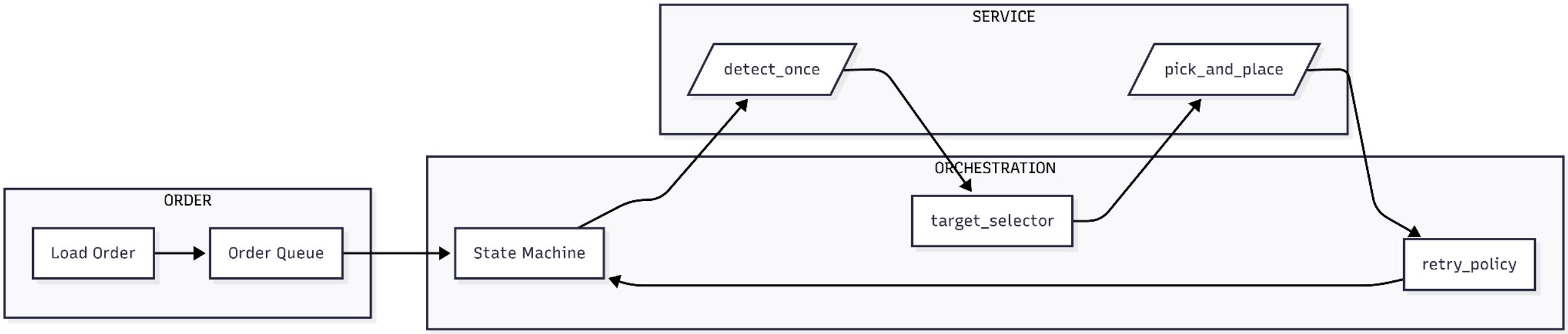
4 종류 모두 pick_and_place 성공



노드 아키텍처


Hardware
task_manager_node
supabase_status_bridge
robot_control_node
perception_transform_node
ROS
RealSense
RG2
Doosan Robot
Arduino Conveyor

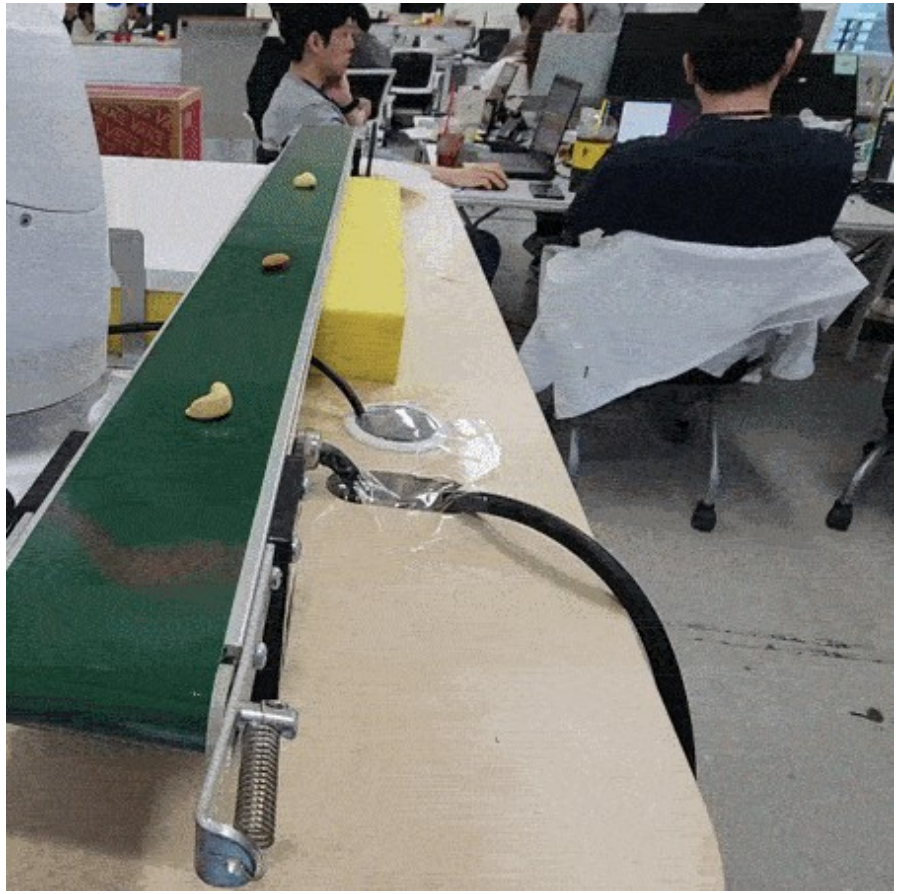
노드 아키텍처 (Task Manager)

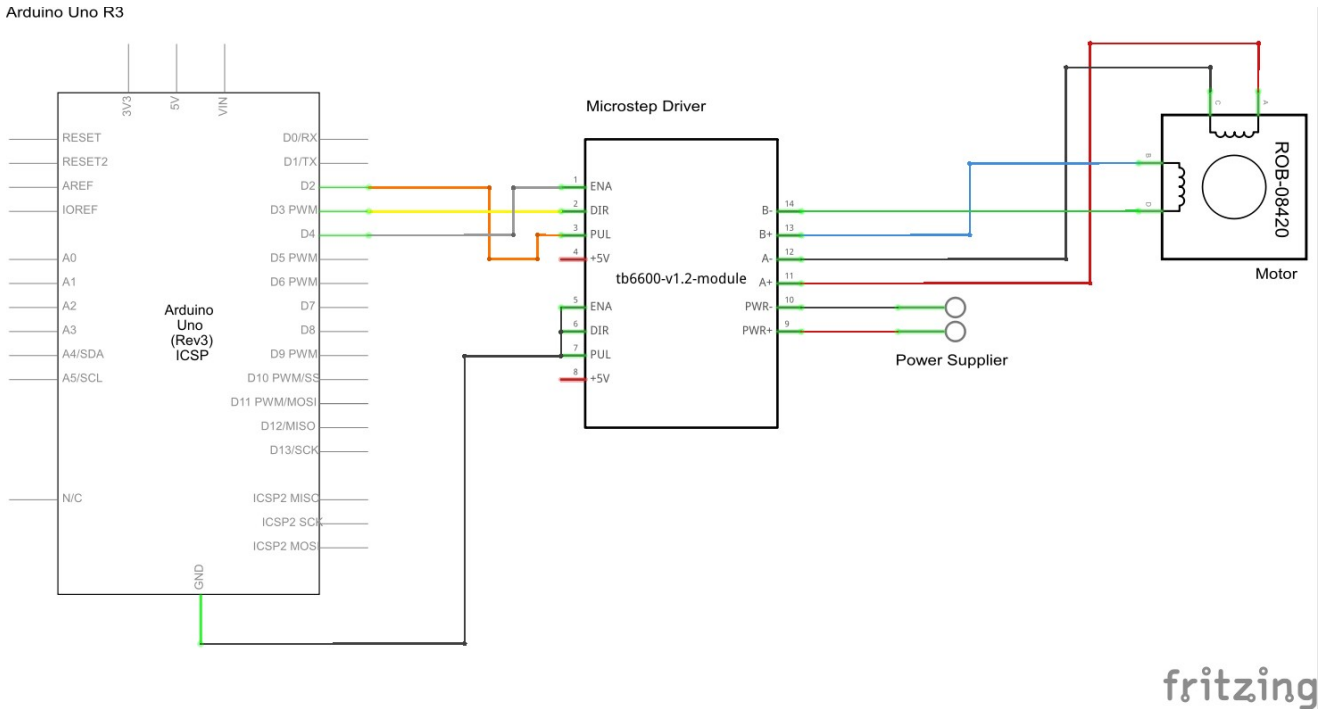


노드 아키텍처

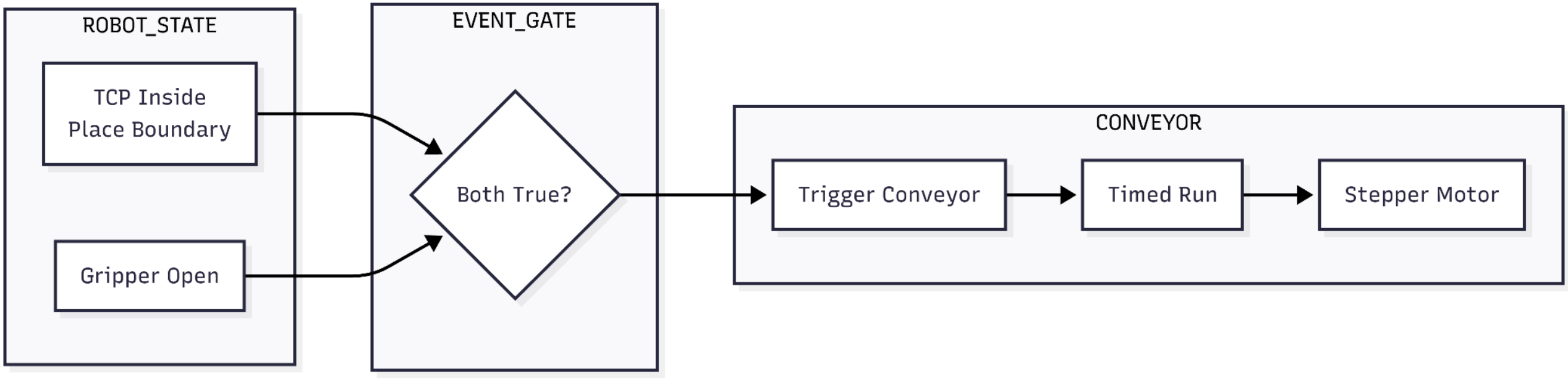

Pick-and-Place 실행
전체 견과류 재탐지
목표 클래스 개수 비교
개수 감소?
Yes
Pick 성공
No
Retry Policy

노드 아키텍처

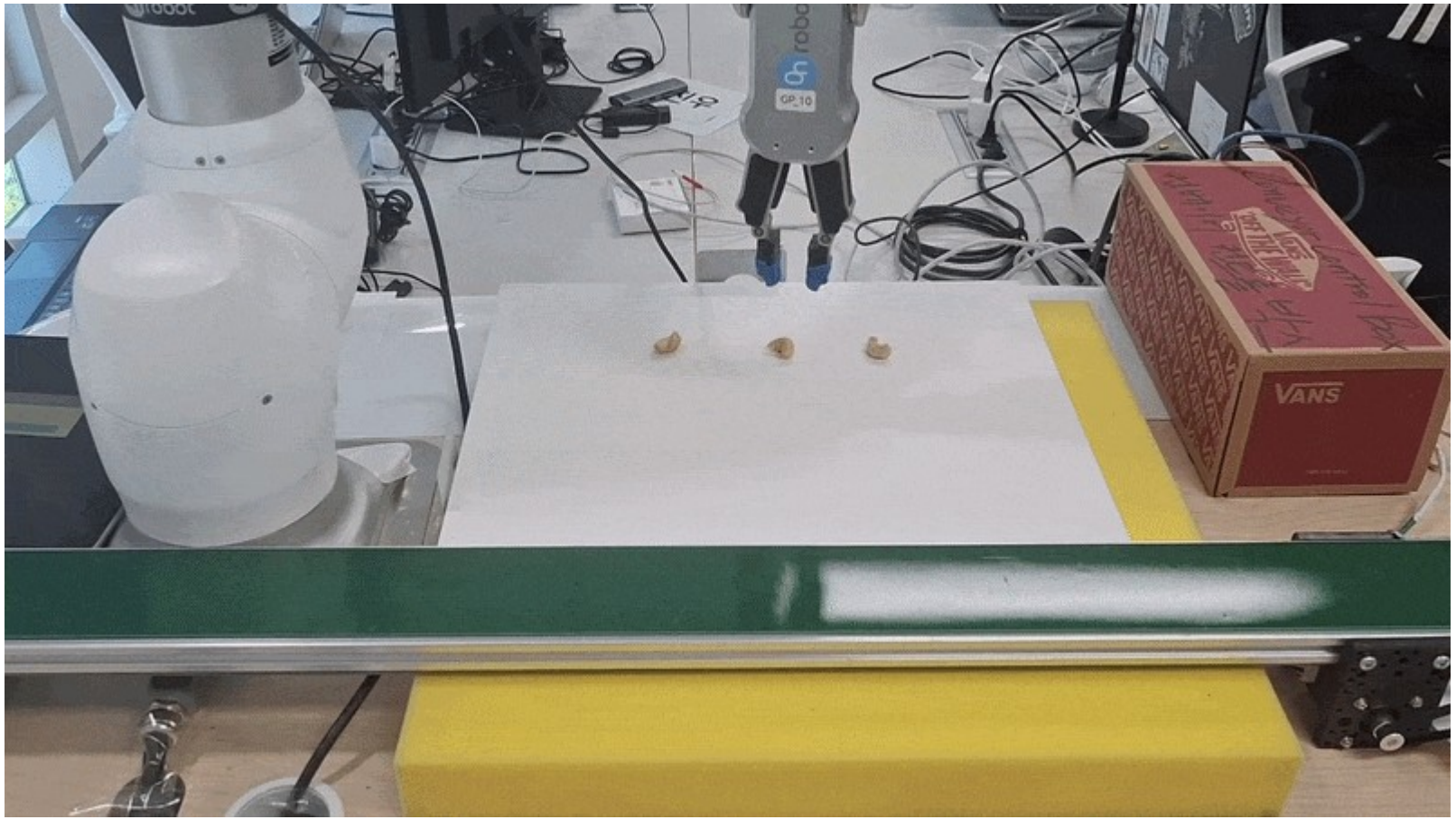



Arduino Uno R3
3V3
5V
VIN
RESET
RESET2
AREF
IOREF
A0
A1
A2
A3
A4/SDA
A5/SCL
N/C
D0/RX
D1/TX
D2
D3 PWM
D4
D5 PWM
D6 PWM
D7
D8
D9 PWM
D10 PWM/SS
D11 PWM/MOSI
D12/MISO
D13/SCK
ICSP2 MISO
ICSP2 SCK
ICSP2 MOSI
GND
Arduino
Uno
(Rev3)
ICSP
Microstep Driver
ENA
DIR
PUL
+5V
ENA
DIR
PUL
+5V
tb6600-v1.2-module
B-
B+
A-
A+
PWR-
PWR+
Power Supplier
ROB-08420
Motor
fritzing

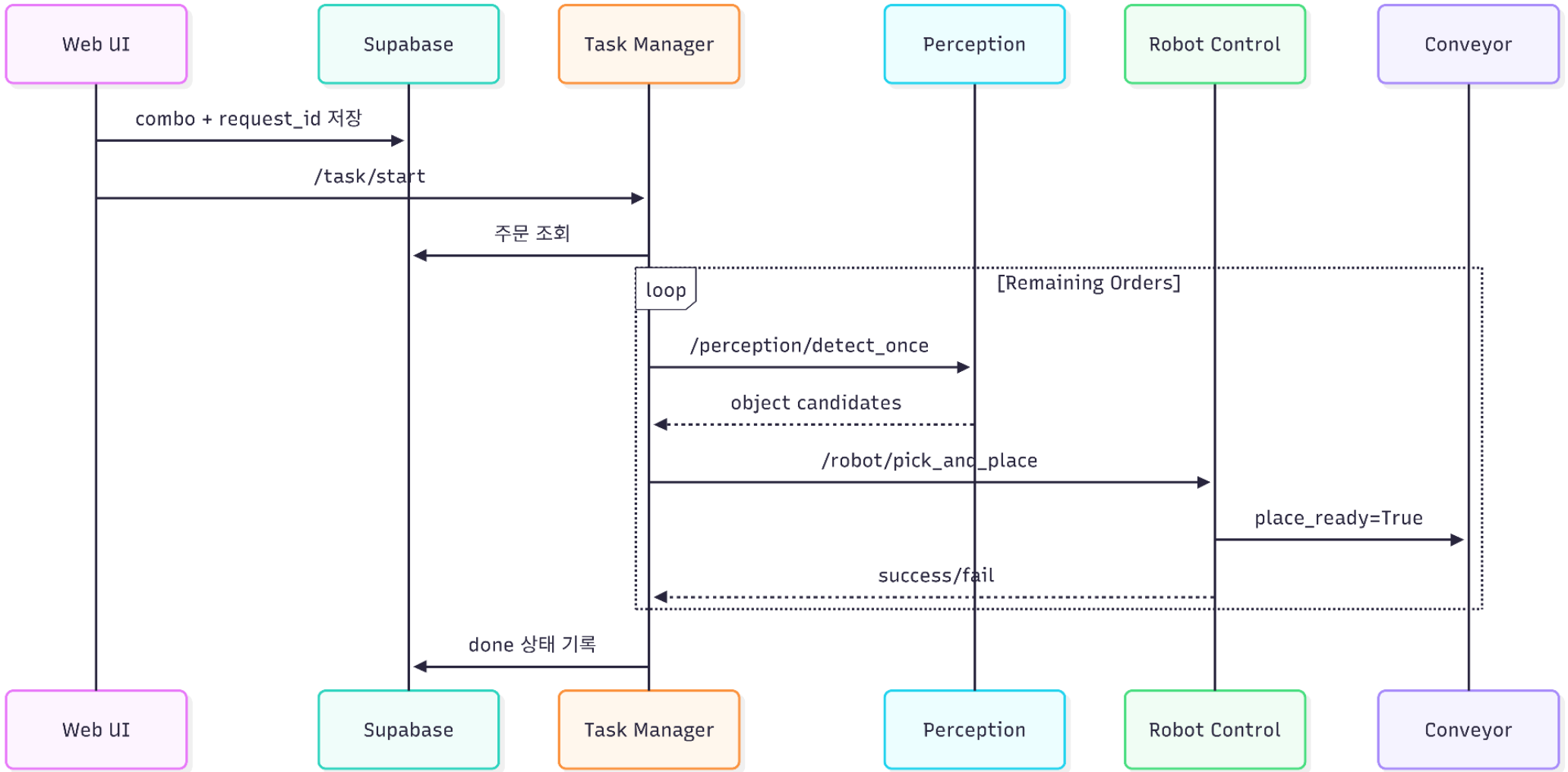
노드 아키텍처


ROBOT_STATE
TCP Inside
Place Boundary
Gripper Open
EVENT_GATE
Both True?
CONVEYOR
Trigger Conveyor
Timed Run
Stepper Motor

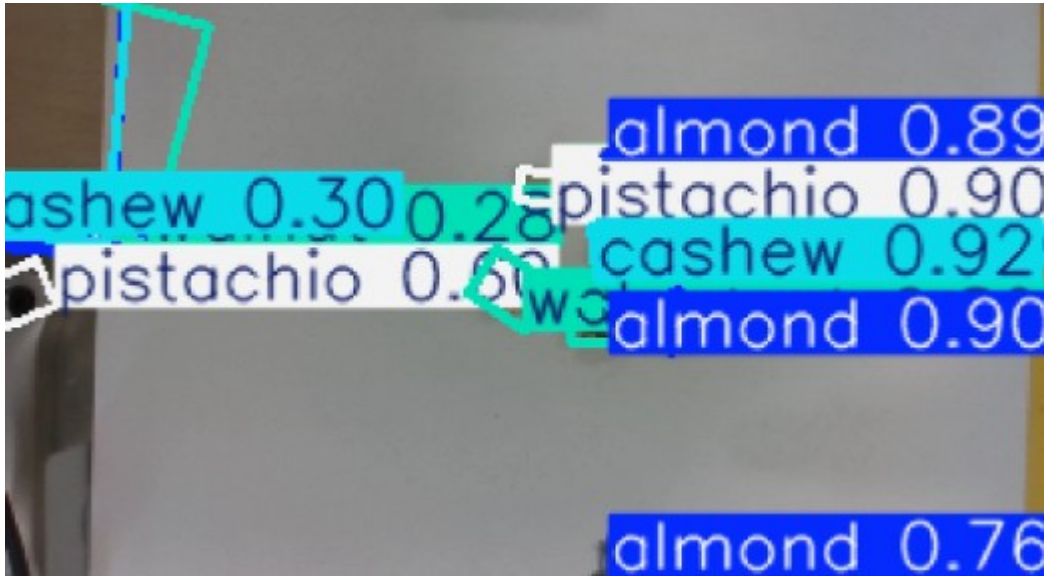
노드 아키텍처

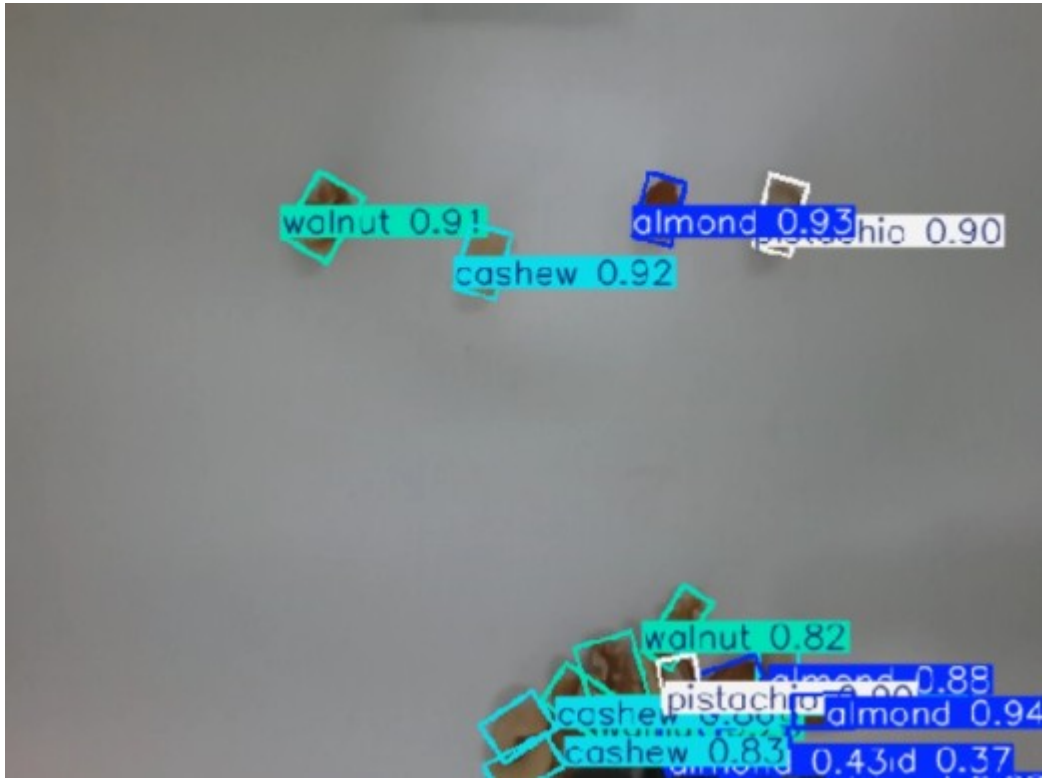


노드 아키텍처 (전체 Runtime 흐름)



오류 이미지


almond 0.89
pistachio 0.90
cashew 0.92
almond 0.90
almond 0.76


walnut 0.91
almond 0.93
cashew 0.92
walnut 0.82
pistachio
almond 0.94
cashew 0.83

오류 이미지



깊이 계산 및 보정

- **깊이 계산 :** OBB 내부 유효 픽셀 median depth 사용
- **Inset 보정 :** OBB 폭 · 높이 0.7 배 축소 후 depth 계산

측정 범위 및 후처리

- **Depth 범위 :** 50 mm ~ 1500 mm
- **Base 후처리 :** depth_offset_mm=-5.0, min_depth_base_mm=2.0

Median 사용 이유

견과류 크기와 모양 다양성으로 인해 생기는 **Depth dropout / outlier** 의 영향 감소

➔ **견과류 pick: 견과류 class 별 z_offset 보정 및 이동을 통한 정확한 파지 구현**

문제 인식 및 목표

- **문제 인식 :** 견과류별 불균일한 모양 , OBB 의 직사각형 형태
- **목표 :** 그리퍼 파지 면적 확대를 위한 위치 변경

그립 전략

- **그립 위치 :** OBB 중심점 (cx, cy) 을 타깃으로 설정
- **그립 Yaw:** 짧은 축을 가로지르도록 정렬

적용 규칙 (Rules)

- **조건 :** width ≥ height
- **각도 :** (θ+90° 혹은 θ)
- **범위 제한 :** [-90°, +90°]

➔ **그리퍼의 축과 견과류 평행 및 위치 정렬**

견과류별 관측 , 파지 보정

견과류별 그립 위치 보정값

class	x_offset	y_offset	z_offset
almond	0.0	-1.0	-2.5
cashew	0.0	-0.5	-2.0
pistachio	0.0	-0.5	-2.0
walnut	0.0	-1.0	-2.0

동일 견과류 구분 최소 거리

클래스	distance_threshold_px
almond	20.0
cashew	20.0
pistachio	20.0
walnut	30.0

Pick 검증 및 보충 수행

문제 정의

info 기존 시스템은 **pick success** 를 실제 이송 성공으로 가정

sensors RG2 grip-detected 는 grasp 순간 **물체 존재**만 의미

warning Transit 중 drop 발생 시 **실제 컨베이어 도착 실패** 가능

➡ **따라서 Pick 당시의 결과와 실제 이송 결과를 분리하여 검증 필요**

Pick 검증 및 보충 수행

알고리즘 요약

analytics 사이클 시작 시 Workspace 견과류 개수 측정

refresh Primary Pick 수행 후 종료 카운트 재측정

calculate 실제 이동량 = 시작 관측 개수 - 종료 관측 개수

inventory_2 부족분 = 주문 개수 - 실제 이동량

replay 부족분이 존재하면 동일 Pick 루프 재실행

task_alt 부족분이 0 이 되거나 최대 라운드 (limit 5 번) 도달 시 종료

예외 처리

보충 이동



견과류 군집 판정

군집 판정 결과를 기반으로 안전한 진입점 (P*) 과 Push 경로 (E) 를 생성

- **목적**

견과류가 서로 밀집되어 Pick 실패 가능성이 있는 상태를 판정

- **입력값**

대상 견과류 중심좌표 T 와 가장 가까운 이웃 중심좌표 N

- **핵심 개념**

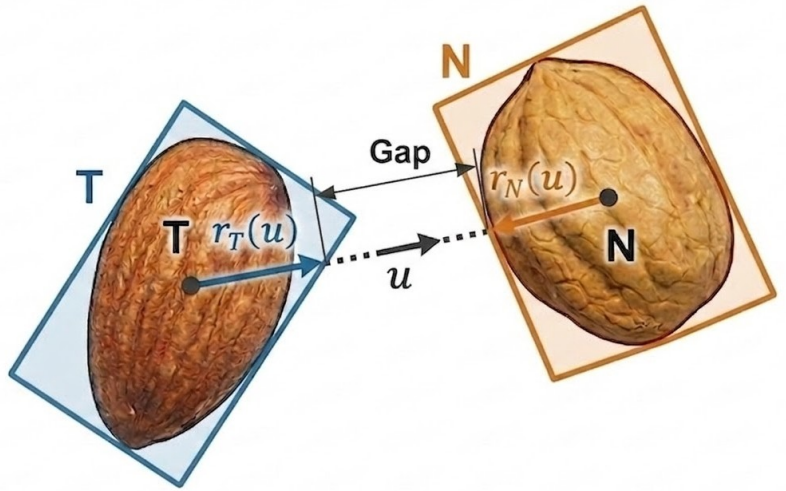
OBB(Oriented Bounding Box) 기반 실제 간격 (gap) 계산

- **판정 기준**

gap < τ 이면 군집 상태로 판단

- **동작**

안전한 진입 방향 계산 후 Push 동작 수행



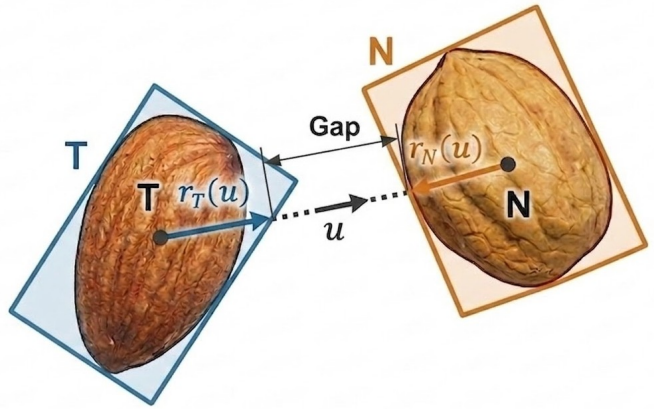
견과류 군집 판정

중심 거리 계산 (두 OBB 중심점 거리)

d = ||N - T||

단위 방향 벡터 (접촉한 OBB 변의 평행)

u = (N - T) / d



OBB 투영 반지름 (두 OBB 의 중심선으로 부터의 변까지의 거리의 합)

r_i(u) = (L_i/2)|cosψ_i·u_x + sinψ_i·u_y| + (S_i/2)|-sinψ_i·u_x + cosψ_i·u_y|

Gap 계산 (OBB 의 변 사이의 거리)

gap(T,N) = d - r_T(u) - r_N(u)

군집 판정 (임계값 τ)

gap(T,N) < τ → Cluster

진입점 생성 (Candidate Entry Point Generation)

목적 : 군집 상태의 견과류를 안전하게 분리하기 위한 접근 방향 계산

- **방향 벡터 계산**
  대상 견과류 (T) 와 이웃 견과류 (N) 를 연결
- **진입 후보 생성**
  연결선에 수직인 방향으로 생성
- **충돌 여부 판단**
  양측 접근 후보 (P_A, P_B) 를 비교
- **최종 진입점 (P*) 선택**
  안전한 방향을 최종적으로 선택



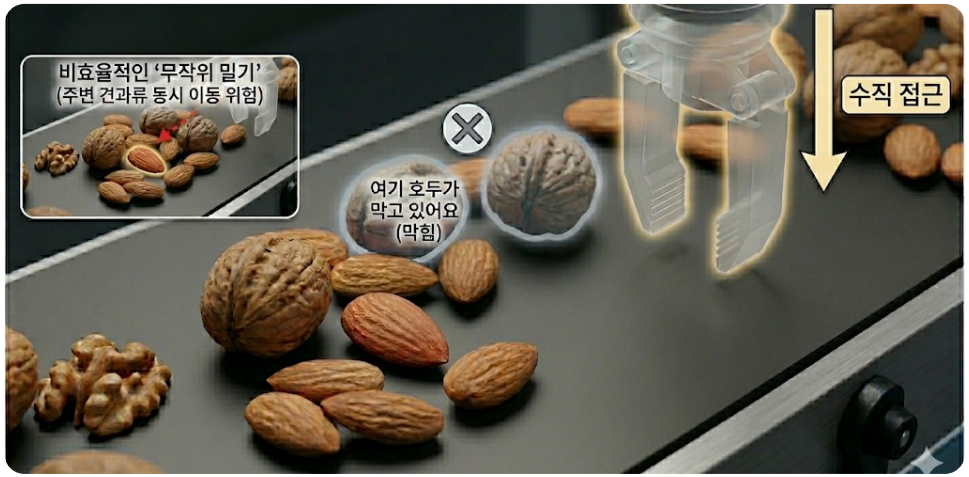
진입점 생성 (Candidate Entry Point Generation)

연결 방향 단위벡터

$u = (N - T) / ||N - T||$

수직 방향 벡터

$a = u\perp = (-u_y, u_x)$



후보 진입점 생성 (견과류의 양측 접근 가능 위치)

$P_A = T + c \cdot a$

$P_B = T - c \cdot a$

의미

대상 견과류 양측에 일정 거리 (c) 만큼 떨어진 접근 후보 위치 생성

후보 진입점 선택 및 충돌 판정

- **후보점 위치 검사**
  후보점이 다른 견과류 내부에 존재하는지 검사
- **충돌 여부 계산**
  OBB 내부 판정을 통해 충돌 여부 계산
- **안전 방향 선택**
  두 후보 중 안전한 방향 선택
- **상단 우선 원칙**
  둘 다 안전하면 상단 (y 값 큰 방향) 우선 선택
- **초군집 판단**
  둘 다 충돌 시 초군집으로 판단

5. 최종 진입점 선택

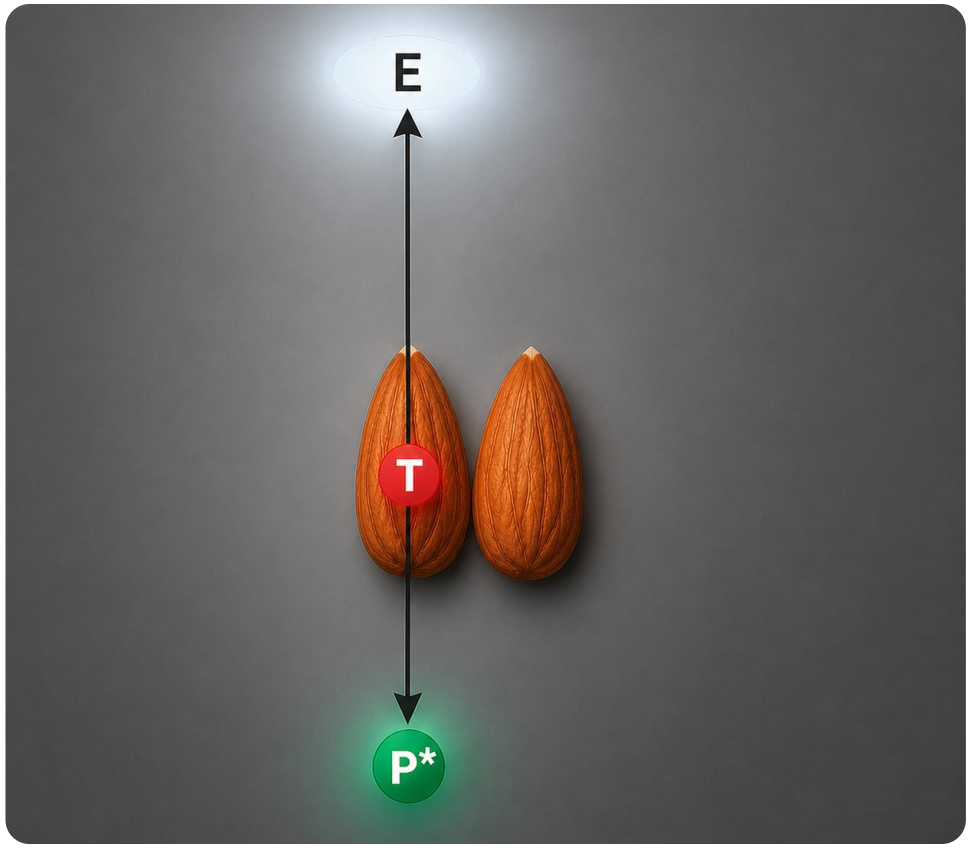
$$P^* = \begin{cases} P_A & \text{if } P_A \text{ 안전, } P_B \text{ 충돌} \\ P_B & \text{if } P_B \text{ 안전, } P_A \text{ 충돌} \\ \arg\max(P_{A,y}, P_{B,y}) & \text{if 둘 다 안전} \\ \varnothing & \text{if 둘 다 충돌} \end{cases}$$

최종 진입점 (P*) 선택 로직

P_A, P_B 두 후보 지점의 안전성 여부를 바탕으로 수식에 따른 최적의 진입점을 결정합니다 .

Push 경로 생성 (Push Path Generation), 이동

- **목적 :** 군집 상태 견과류를 분리하기 위한 Push 이동 경로 생성
- **선택된 진입점 (P*) 을 기준으로** Push 방향 계산
- **대상 견과류 중심 (T) 을 지나** 반대 방향으로 이동
- **Push 거리 배율 (k) 을 적용하여** 종료점 (E) 생성
- **직선 Push 경로를 통해** 견과류 간 간격 확보
- **홈으로 정렬후 재관측 ,** 군집이 유지시 해당 작업 반복



알고리즘 요약 : 진입점 (P*), 중심 (T), 종료점 (E) 을 이용한 직선 이동을 통해 군집 해소

Push 경로 생성 (Push Path Generation)

진입 변위 벡터

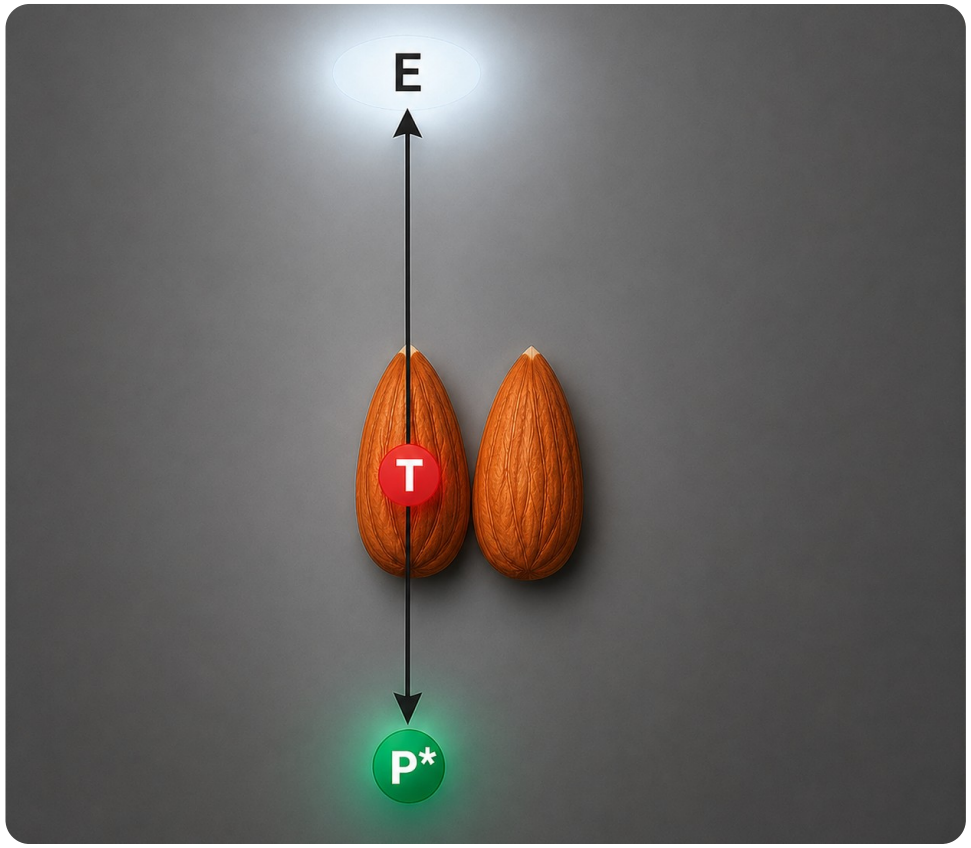
$$\Delta = P^* - T$$

Push 종료점 계산

$$E = T - k(P^* - T)$$

좌표 형태

$$E_x = t_x - k(P^*_x - t_x)$$
$$E_y = t_y - k(P^*_y - t_y)$$



기하적 의미 : P*(진입점) , T(대상 견과류 위치) , E(최종 목표 지역) 는 동일 직선 위에 위치

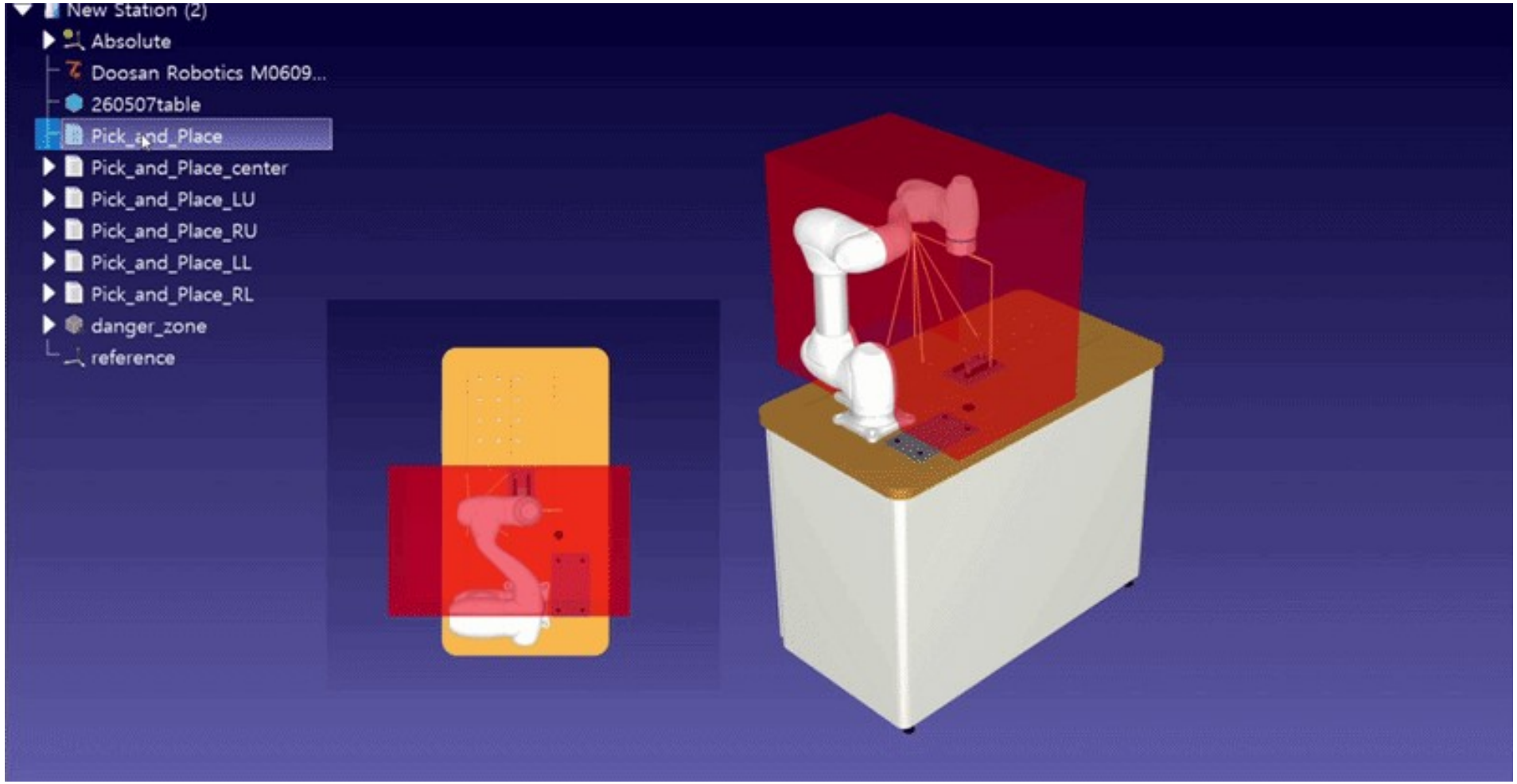
예외 처리

군집 분산



안전영역

RoboDK CAD 활용



주요 산출물 — 패키지 · 모델 · UI · 인프라 · 문서

본 발표 시점에 공개된 결과물 다섯 가지

11

ROS 2 패키지

bringup / voice / task_manager / object_detection / perception / robot_control / conveyor 등

1

YOLO best.pt

4 클래스 · test mAP@0.5 = 0.992 · 추론 ~15ms · OBB 회전 검출

2

React 웹 UI

v1 web_stt_firebase (Firebase 미러) · v2 풀 - 브라우저 시연 대상

1

Firestore 문서

robot_session/current — 음성 + 로봇 실시간 미러 , merge:true 갱신

8

운영 문서

01 시스템 아키텍처 / 02 노드 / 03 안전 / 04 운영 매뉴얼 등

운영 측면 핵심 산출

- 단일 launch(full_system.launch.py) 로 mock·real 양 모드 부팅
- Firestore 끊겨도 로봇 파이프라인은 영향 없음 (graceful degradation)
- OBB 좌표 클리핑 ·multi-frame fusion 등 데이터 안정성 보강 코드 포함

자체 평가 의견 — 완성도 · 개선점 · 잘한 점 · 느낀 점

프로젝트 기획 의도와의 부합 정도 , 실무 활용 가능성 , 달성도 등 자체 평가

프로젝트 결과물 완성도 평가

9.9 / 10 점

- 메인 기능 구현이 일찍 끝나서 챌린지와 예외 처리 등 완성도에 많은 시간을 투자할 수 있었다 .
- yolov8 OBB 를 활용한 탐지와 좌표를 기반으로한 여러 함수와 알고리즘을 사용하여 복잡한 기능의 구현이 가능했다 .
- 단기간에 목표에 가까운 정밀한 견과류 객체 탐지 모델을 완성시켜서 통합 개발을 빠르게 진행 할 수 있었다 .

잘한 부분과 아쉬운 점 (협업)

9.9 / 10 점

- 구현을 우선 하다보니 진행 사항이 제대로 공유되지 않을 때가 있어서 아쉬웠다 . 그만큼 메인 기능 구현을 빨리 완성해서 완성도를 높인 점은 잘했다고 생각한다 .
- 임시 통합 과정에서 폴더 경로를 통일하지 못하여 오류가 발생하며 시간이 지체되었다 . 사전에 협의를 하지 못한 부분이 아쉬웠다 . 수학 함수를 이용하여 복잡한 움직임을 구현하여 잘만들었다 .
- 더 많은 데이터셋과 규모가 큰 학습 모델로 학습하면 정밀한 결과를 도출할 수 있었는데 시간적 제약 때문에 못했던 것이 아쉬웠다 . 그럼에도 불구하고 단기간에 매우 높은 정확도를 가진 객체탐지 모델을 만든것은 잘했다 .
- 시간만 더 있었다면 , Docker 로 Node 여러개 나눠서 작동하는거 보고싶다...

자체 평가 의견 — 완성도 · 개선점 · 잘한 점 · 느낀 점

프로젝트 기획 의도와의 부합 정도 , 실무 활용 가능성 , 달성도 등 자체 평가

느낀 점 · 경험한 성과 (경력 계획 연관)

9.9 / 10 점

• Docker 를 써볼 수 있었고 , 모델 학습을 프로젝트에서 써볼 수 있었던 점이 의미 있었다고 생각한다 .

• 로봇 모션 동작 시나리오를 만들고 수학 함수를 이용하여 복잡한 움직임을 구현하여 만족스러웠다 .

• Frontend 영역과 RDBMS + PostgreSQL 활용 재미났다 . firebase 에서 supabase 로 전환하는 과정에서 rosbridge 의 기능및 능력을 확인할 수 있었다 . 컨베이어 고치기랑 Confusion Matrix 분석에 따라서 적절한 사진 모으기도 재미있었다 . ㅎㅎ

추후 개선점 · 보완할 점 (기술적 측면)

9.9 / 10 점

• 현재는 DB 를 통해 재고를 추적하고 있는데 , 후에 재고 부족 알림이 뜨는 등 실제 기능과 연결되면 좋겠다고 생각한다 .

• 지금은 견과류 class 마다 보정값을 부여하여 정확도를 높였지만 , 종류와 크기에 따라 다른 함수를 적용하도록 만드는 것이다 .

• Website 에 있는 울렁이는 3D 지형에 , 실시간으로 Realsense 로부터 하얀색 도화지 (평평한 Base) 에 놓여진 견과류들의 좌표값들이나 Depth 등을 시각화 하고싶다… 지금 3D Terrain 은 5 dimensional parametric data visualization 가능이다… 마치 인공위성으로 지형 스캔본을 GUI 로 볼 수 있는 것처럼… Geomatics!

최종 영상



마치며 ..

